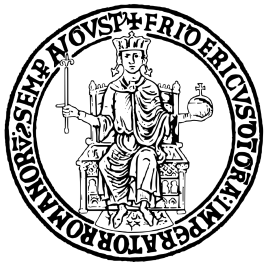
UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II



SCUOLA POLITECNICA E DELLE SCIENZE DI BASE

DIPARTIMENTO DI INGEGNERIA ELETTRICA E DELLE

TECNOLOGIE DELL'INFORMAZIONE

CORSO DI LAUREA IN INFORMATICA

TESI SPERIMENTALE DI LAUREA TRIENNALE

# *PERSONALIZZAZIONE DEI COMPORTAMENTI ROBOTICI IN BASE ALLO SCHEDULING DELLE ATTIVITÀ QUOTIDIANE DEGLI UTENTI*

**Relatrice**
Prof. ssa Alessandra ROSSI

**Candidato**
Fabiola SALOMONE
N86002870

Anno Accademico - 2023-2024

UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

SCUOLA POLITECNICA E DELLE SCIENZE DI BASE

DIPARTIMENTO DI INGEGNERIA ELETTRICA E DELLE

TECNOLOGIE DELL'INFORMAZIONE

CORSO DI LAUREA IN INFORMATICA

TESI SPERIMENTALE DI LAUREA TRIENNALE

# *PERSONALIZZAZIONE DEI COMPORTAMENTI ROBOTICI IN BASE ALLO SCHEDULING DELLE ATTIVITÀ QUOTIDIANE DEGLI UTENTI*

**Relatrice**
Prof. ssa Alessandra ROSSI

**Candidato**
Fabiola SALOMONE
N86002870

Anno Accademico - 2023-2024

# INDICE

1. Introduzione.....7
1.1 Progetto.....9
1.2 Obiettivo della tesi.....11
1.3 Struttura Tesi.....12
2. Materiali e Tecnologie.....14
2.1 Temi Robot.....14
2.1.1 Temi SDK.....18
2.1.2 Struttura Temi.....20
2.1.3 Sistema di navigazione ROBOX™.....34
2.1.4 Temi OS.....37
2.2 Ambiente Android.....39
2.2.1 Android Studio.....41
2.2.2 Android Debug Bridge per la connessione a Temi.....44
2.2.3 Room Database.....45
2.3 Python.....47
2.3.1 Libreria : face_recognition.....48
3. Approccio proposto.....50
3.1 Analisi del comportamento del robot.....50
3.1.1 Interazione con l'Utente.....50
3.1.2 Navigazione e Movimento del robot.....53
3.1.3 Rilevamento dell'utente.....55
3.1.4 Riconoscimento dell'utente.....56
3.2 Progettazione dell'applicazione.....57
3.2.1 Schermata "AUTHENTICATION".....58

3.2.2 Schermata “HOME” ..... 62
3.2.3 Schermata “CREATE ROUTE” ..... 63
3.2.4 Schermata “START PATROL” ..... 65
3.2.5 Schermata “OPTIONS ACTIVITY” ..... 66
3.2.6 Schermata “ACTIVITY NOTIFICATION” ..... 70
3.2.7 Schermata “Option Location Or Point Interest” ..... 72
3.2.8 Schermata “Insert Location Or Point Interest” ..... 72
3.2.9 Schermata “Delete Location Or Point Interest” ..... 74
3.2.10 Schermata “Edit Location Or Point Interest” ..... 76
3.2.11 Schermata “Execution tasks” ..... 78
4. Architettura dell’applicazione ..... 79
4.1 Implementazione dell’applicazione ..... 79
4.1.1 Architettura Esterna ..... 79
4.1.2 Architettura Client ..... 81
5. Implementazione ..... 87
5.1 Dettagli Implementativi ..... 87
5.1.1 Autenticazione ..... 88
5.1.2 Creazione e Visualizzazione dei percorsi ..... 89
5.1.3 Inserimento locazioni o punti d’interesse ..... 91
5.1.4 Eliminazione locazione o punto d’interesse ..... 92
5.1.5 Modifica locazione o punto d’interesse ..... 93
5.1.6 Inserimento e Visualizzazione attività ..... 94
5.1.7 Notifica attività ..... 95
6. Testing ..... 99
6.1 Descrizione dei test ..... 99
6.2 Valutazione ..... 101
6.3 Risultati ..... 105

7. Conclusioni e lavori futuri.....107
Bibliografia e Sitografia .....110

# 1. Introduzione

Nell'era dell'innovazione tecnologica, la robotica e l'intelligenza artificiale hanno dato vita a una nuova generazione di robot progettati per esercitare le mansioni di assistenti personali, con l'obiettivo di semplificare la vita quotidiana delle persone e diventare compagni utili e interattivi all'interno delle abitazioni e degli ambienti sociali.

In particolare i robot destinati agli ambienti domestici stanno emergendo come una soluzione promettente sia dal punto di vista dei costi che dell'assistenza, in risposta ai cambiamenti demografici caratterizzati dall'invecchiamento della popolazione. Questi robot di servizio non solo forniscono assistenza agli anziani, ma li motivano anche e offrono un supporto attivo per la riabilitazione. Questo approccio può essere descritto come “supportare le persone a fare" anziché “fare a/per le persone". In pratica, il robot di assistenza collabora in modo continuo con la persona anziana, adattandosi alle mutevoli esigenze legate all'invecchiamento. Di conseguenza, il robot diventa un compagno fidato per la persona anziana [1].

Inoltre, questi robot dimostrano anche un'elevata efficienza nell'interazione con i bambini in quanto sono in grado di coinvolgere i bambini in una serie di attività, tra cui il riordino della camera, il completamento dei compiti, l'instaurazione di una routine per l'igiene personale e molto altro [2].

Nello specifico, tali robot intelligenti e interattivi forniscono una vasta gamma di servizi e funzioni utili come:

- **Gestione delle informazioni** : Grazie alla connessione a Internet, i robot assistenti possono fornire informazioni in tempo reale su una varietà di argomenti, come notizie, previsioni del tempo, risultati sportivi e altro ancora;

- **Organizzazione degli impegni** : Questi robot possono aiutare a tenere traccia degli appuntamenti, delle scadenze e degli eventi importanti, impostando promemoria e allarmi;

- **Controllo domestico** : I robot assistenti personali possono integrarsi con dispositivi smart home, consentendo agli utenti di controllare luci, termostati, elettrodomestici e sistemi di sicurezza con semplici comandi vocali;

- **Assistenza alla mobilità** : Alcuni modelli di robot sono dotati di capacità di movimento e possono aiutare nelle attività di trasporto di oggetti leggeri o di recuperare oggetti da altre stanze;

- **Intrattenimento** : I robot assistenti possono fornire intrattenimento agli utenti, riproducendo musica, film, serie TV o persino giocare a giochi interattivi;

- **Apprendimento e personalizzazione** : Questi robot sono in grado di apprendere dalle interazioni con gli utenti e adattarsi alle loro preferenze personali, offrendo un'esperienza sempre più personalizzata e soddisfacente;

- **Assistenza emotiva** : Alcuni modelli di robot assistenti sono stati progettati per offrire supporto emotivo e compagnia agli utenti, particolarmente utili per le persone anziane o in situazioni di solitudine.

In conclusione, i robot destinati agli ambienti domestici, non sono solo macchine, ma veri e propri compagni di vita, capaci di apprendere dalle interazioni con gli utenti e di adattarsi alle loro preferenze personali [1].

In questa tesi, ci concentreremo sulla progettazione e implementazione di un'applicazione progettata con l'obiettivo di rendere il robot un'assistente personale ancora più versatile e funzionale nelle sue interazioni con gli utenti. Esploreremo gli aspetti chiave della creazione di un'applicazione per il robot Temi, analizzando le sfide e le opportunità che questa avventura offre.

Attraverso questa tesi, si cercherà di fornire una guida dettagliata per la realizzazione di un'applicazione per il robot Temi, con un'analisi completa delle tecnologie coinvolte, dei framework di sviluppo e delle best practice per ottimizzare l'esperienza dell'utente.

## 1.1 Progetto

Il progetto, oggetto della presente tesi, riguarda lo sviluppo di un'applicazione Android per il robot Temi. Tale applicazione ha lo scopo di offrire agli utenti la pianificazione dei propri impegni che includono ad esempio il tener traccia degli appuntamenti, delle scadenze e degli eventi importanti, impostando promemoria e allarmi.

L'applicazione sarà, fondamentalmente, divisa in cinque parti:

1. Una prima parte dedicata alla registrazione e all'autenticazione dell'utente : in per fase di registrazione l'utente può registrarsi all'applicazione per poi usufruire delle diverse funzionalità inserendo : un nome, un l'email, una password e la foto del volto. Mentre per l'accesso è necessario che l'utente inserisca email e password inoltre qualora l'utente dimentica la password o la smarrisce non occorrerà registrarsi nuovamente ma può cambiarla tramite l'operazione di password dimenticata.

2. Una seconda parte dedicata all'inserimento, alla cancellazione e alla modifica di nuove locazioni o punti d'interesse sulla mappa del robot. L'utente potrà inserire, cancellare e modificare una locazione o un punto d'interesse sulla mappa interna del robot Temi.

3. Una terza parte dedicata all'inserimento, alla cancellazione e alla modifica di percorsi. Attraverso ciò l'utente, autenticato, può creare, cancellare e modificare un percorso. Il percorso viene creato da utenti compilando un form in cui dovrà inserire un nome e una o più locazione o punto d'interesse, che permetteranno di creare un solo o più percorsi che il robot in futuro dovrà intraprendere. Tutti i percorsi presenti saranno registrati in una lista e ognuno potrà essere cancellato, modificato o eseguito.

4. Una quarta parte dedicata all'inserimento , alla cancellazione e alla modifica di una attività . L'utente avrà la possibilità di inserire e cancellare una attività mediante un form inserendo il nome dell'attività, scegliendo una locazione (come ad esempio cucina, ufficio, bagno, camera da letto o altro), dettagliando una specifica attività, indicando una fascia oraria e infine indicando il giorno in cui il robot dovrà ricordare l'attività all'utente.

5. Una quinta parte dedicata all'attivazione dell'attività. Vengono collezionati un elenco di attività per l'utente in diverse posizioni e nel momento in cui una precisa attività viene attivata, dall'utente, il robot Temi va alla ricerca dell'utente nella posizione indicata e se l'utente viene rilevato, il robot Temi avvia il riconoscimento e in caso di esito positivo viene notificata l'attività pianificata. In caso contrario, cioè quando l'utente non viene rilevato in una particolare locazione o punto d'interesse l'attività viene riprogrammata automaticamente dopo 5 minuti, invece se l'utente non viene riconosciuto il robot torna alla *docking station*.

6. Una sesta parte dedicata all'esecuzione delle attività. Vengono eseguite le attività in base all'orario corrente.

## 1.2 Obiettivo della tesi

L'obiettivo principale del progetto è quello di sviluppare un'applicazione user-friendly per utenti non esperti al fine di semplificare e ottimizzare la gestione delle loro attività quotidiane. Questa applicazione consentirà agli utenti di pianificare, organizzare e monitorare in modo efficiente le proprie attività e impegni. Inoltre, si mira a utilizzare il robot Temi per facilitare l'esecuzione di alcune di queste attività, migliorando così l'efficienza e il completamento di tali attività .

In particolare sarà definito un form che consente di registrare una serie di informazioni, relative all'attività, in modo tale che ad un particolare istante, in base all'orario effettivo, il robot si porterà alla locazione salvata e se riconosce l'utente notificherà l'attività. Al tempo stesso si vuole realizzare un interfaccia che faccia in modo che quanto detto risulti di facile comprensione e permetta una semplice interazione per l'utente. Tale operazione deve essere di tipo multi-utente, il che significa che si avranno

differenti profili di utenti che programmano le proprie attività sulla base degli slot temporali disponibili.

Il sistema dovrà dare la possibilità all'utente di creare un percorso tramite un form e di visualizzare la lista di percorsi.

Ogni percorso può essere modificato, eliminato o eseguito.

Infine, ci si è posti come obiettivo quello di programmare il sistema al fine di consentire l'inserimento, l'eliminazione e l'aggiornamento di nuove locazioni o punti d'interesse sulla mappa presente all'interno del sistema robot.

## 1.3 Struttura Tesi

La tesi sarà composta da sette capitoli :

- Capitolo 1 "Introduzione", verrà presentata una breve introduzione dell'argomento di studio dell'intera tesi.

- Capitolo 2 "Materiali e tecnologie", verranno presentati gli strumenti e le tecnologie utilizzate per la realizzazione del progetto. Come ad esempio il robot Temi, le API, l'SDK Temi, il database e molto altro.

- Capitolo 3 "Approccio proposto", verranno illustrati aspetti relativi al comportamento del robot e gli aspetti legati alla fase di progettazione del modulo realizzato.

- Capitolo 4 “Architettura dell’applicazione”, verrà illustrata l’architettura dell’applicazione dettagliando l’architettura esterna e l’architettura del client del modulo realizzato.

- Capitolo 5 “Implementazione”, dove verranno presentati i dettagli relativi agli aspetti implementativi del modulo realizzato.

- Capitolo 6 “Testing”, dove verrà presentato il processo di validazione del modulo realizzato.

- Capitolo 7 “Conclusioni e lavori futuri” verranno illustrate le conclusione sullo studio della tesi e saranno proposte delle idee per possibili lavori futuri.

# 2. Materiali e Tecnologie

In questo capitolo, vengono illustrate e descritte le tecnologie utilizzate per la realizzazione del progetto descritto nel capitolo dell'approccio proposto. In particolare, a ricoprire un ruolo fondamentale è Temi, *Android studio*, *Room Database* e *Python*.

## 2.1 Temi Robot

Temi è un avanzato e interattivo robot assistente personale progettato e sviluppato dall'azienda Robotemi Inc[1]. Temi è attualmente utilizzato per semplificare la vita quotidiana delle persone e diventare un compagno utile e interattivo all'interno delle abitazioni e degli ambienti sociali.

Il robot è dotato di un sofisticato sistema di intelligenza artificiale, che gli consente di comprendere i comandi vocali, elaborare il linguaggio naturale e apprendere dalle interazioni con gli utenti. Ciò gli permette di fornire risposte e informazioni precise, gestire promemoria, impegni e controllare dispositivi smart home, tra le tante altre funzioni.

Grazie alla sua struttura mobile, Temi può muoversi autonomamente e navigare in modo sicuro all'interno degli ambienti, evitando ostacoli e adattandosi alle dinamiche degli spazi circostanti. Questo lo rende estremamente versatile e adatto ad affrontare una vasta gamma di attività.

[1] https://www.robotemi.com/

Una delle caratteristiche distintive di Temi è la sua capacità di essere un compagno socievole e di intrattenimento. Può riprodurre musica, film e programmi TV, intrattenere ospiti con giochi interattivi e coinvolgenti e fornire un supporto emotivo a persone anziane o in situazioni di solitudine.

Il robot gode di una vera è propria rete (Figura 1.1) attraverso il quale i differenti robot possono condividere informazioni, dati e comandi, consentendo loro di lavorare in modo cooperativo e interagire con l'ambiente circostante e con le persone. La rete di Temi può essere composta da diverse componenti, tra cui:

- **Temi Cloud** : una piattaforma basata su cloud fornita da Robotemi Inc. che funge da server centrale per gestire i dati e le comunicazioni tra i robot e altri dispositivi. Il Temi Cloud supporta funzionalità come il tracciamento della posizione dei robot, la gestione delle autorizzazioni relative all'utente e l'invio di aggiornamenti software.

- **Connessione Internet** : i robot Temi hanno bisogno di una connessione Internet per accedere al Temi Cloud e stabilire comunicazioni con altri dispositivi o servizi online. La connessione Internet consente ai robot di ricevere aggiornamenti software, caricare dati, e accedere a risorse e informazioni online.

- **Temi Robot-to-Robot Communication** : i robot Temi possono comunicare tra loro all'interno della rete, permettendo loro di scambiarsi dati e informazioni. Questa comunicazione può essere utilizzata per la collaborazione tra robot o per fornire supporto reciproco durante le attività.

- **Temi Client Communication** : i robot Temi possono comunicare con altre applicazioni o dispositivi tramite le interfacce offerte dal Temi SDK (*Software Development Kit*) o tramite API (*Application Programming Interface*). Questa comunicazione consente agli sviluppatori di creare applicazioni personalizzate per i robot e di integrarli in diversi contesti e servizi.

- **Comunicazione con Utenti** : i robot Temi sono dotati di funzionalità di interazione con gli utenti, come riconoscimento vocale e capacità di risposta vocale. Questa comunicazione permette agli utenti di interagire con i Robot in modo naturale e intuitivo.

La rete di Temi è un sistema completo che permette ai robot di comunicare tra loro, con il Temi Cloud e con altre applicazioni o dispositivi. Questa connettività avanzata consente ai robot Temi di offrire una vasta gamma di funzionalità e di diventare un compagno utile e interattivo all'interno delle abitazioni e degli ambienti sociali.

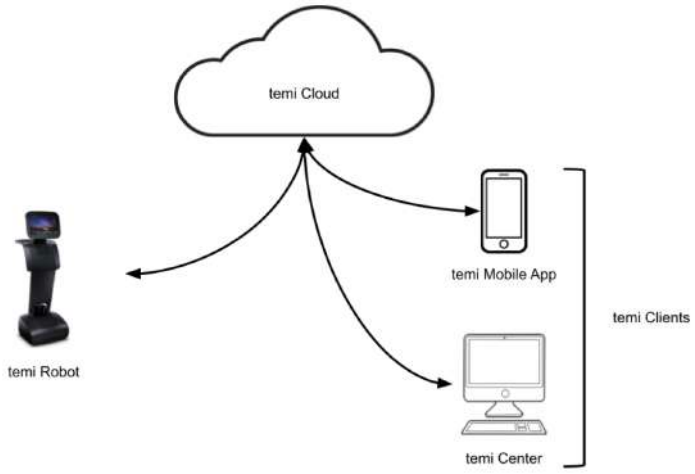



Figura 1.1: architettura rete di Temi [3]

Nello schema sopra:

- **Temi robot** : un robot con navigazione autonoma e funzionalità di telepresenza remota.

- **Temi Mobile App** : un’app per smartphone (iOS e Android) che può avviare una sessione di telepresenza con temi Robot.

- **Temi Center** : un'applicazione browser Web in grado di gestire e avviare in remoto sessioni di telepresenza con temi Robot.

- **Temi Cloud** : Un insieme di servizi web. Il suo scopo principale è archiviare le informazioni dell'utente e assistere nella creazione di sessioni di telepresenza tra temi Robot e temi Mobile o temi Center.

Questa rete, quindi, consente ai Temi di ricevere istruzioni e comandi dall'utente tramite il proprio *smartphone* o altre applicazioni, interagire tra di loro per supportarsi reciprocamente o condividere informazioni, e accedere ai servizi forniti dal Temi Cloud per gestire dati, aggiornamenti software e altre funzionalità avanzate. La connettività avanzata tra i robot temi e il cloud permette loro di fornire una navigazione autonoma, interagire con gli utenti in modo intelligente e offrire una vasta gamma di funzionalità per semplificare la vita quotidiana delle persone.

### 2.1.1 Temi SDK

Per lo sviluppano di applicazioni in Temi, l'azienda Robotemi Inc. offre un'insieme di strumenti, librerie, documentazione e risorse per consentire agli sviluppatori di creare applicazioni personalizzate e interagire con il robot Temi.

L'SDK di Temi, messa a disposizione dall'azienda, offre agli sviluppatori tutte le risorse necessarie per sfruttare le funzionalità avanzate del robot, come il controllo del movimento, il riconoscimento vocale, il riconoscimento facciale, l'elaborazione del linguaggio naturale e molto altro ancora.

Attraverso l'SDK, è possibile creare applicazioni innovative, giochi, assistenti vocali personalizzati e altre funzionalità che arricchiscono l'esperienza degli utenti con il robot. Le componenti chiave che vengono incluse nell' SDK di Temi includono:

**API** (*Application Programming Interface*)

Interfacce di programmazione che consentono agli sviluppatori di comunicare con il robot e accedere alle sue funzionalità.

**Documentazione**

Guida dettagliata che spiega le funzionalità del robot, come utilizzare le API e i comandi disponibili, e fornisce esempi di codice per aiutare gli sviluppatori a iniziare.

**Librerie e strumenti di sviluppo**

Strumenti e librerie specifici che semplificano lo sviluppo di applicazioni per il

robot Temi.

**Supporto per integrazione di terze parti**

Possibilità di integrare servizi o piattaforme di terze parti nell'applicazione.

Tuttavia non include:

**Simulatori**

Strumenti che consentono agli sviluppatori di testare e verificare le loro applicazioni senza doverle eseguire sul robot reale.

Quindi, quando si sviluppano applicazioni in Temi, è possibile provarle solo direttamente sul robot fisico ma non su un robot virtuale o su un simulatore.

Per lo sviluppo, è stata introdotta la libreria:

1. implementation 'com.robotemi:sdk:1.131.3'

Essenziale per creare applicazioni e funzionalità personalizzate per il robot Temi, tra cui il controllo dei movimenti, l'interazione vocale e altre funzionalità.

## 2.1.2 Struttura Temi

Temi presenta una dimensione (figura 1.3) di 3.2' ft. / 100 cm. x 1.1' ft. / 35 cm. x 1.5' ft. / 45 cm. e un peso di  26 lbs / 12 kg.




Figura 1.3: Dimensione di Temi

È costituito dalle seguenti componenti:

- **Schermo LCD HD da 13.3”** : touch screen dotato di rivestimento resistente alle impronte digitali e con tecnologia capacitiva, per un'interfaccia intuitiva, e resistente, inoltre gode di display di pixel di densità 224 (ppi) e luminosità massima di 340 cd/m2.

  Inoltre lo schermo è dotato di inclinazione (figura 1.4) con gamma di movimento 15°~+55° per il rilevamento del volto autonomo dove grazie al motore CC senza spazzole con riduttore epicicloidale fa si che il rilevamento sia quanto più preciso possibile e fa risultare il funzionamento silenzioso.

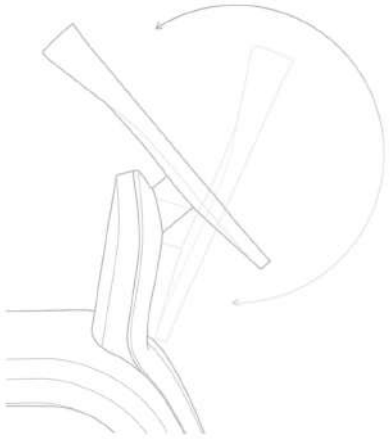


Figura 1.4: Inclinazione schermo

- **quattro microfoni** (figura 1.5) : digitali omnidirezionali per il rilevamento della voce. Supporta la localizzazione in tempo reale, formatura del raggio, cancellazione acustica dell'eco e riduzione del rumore ambientale.

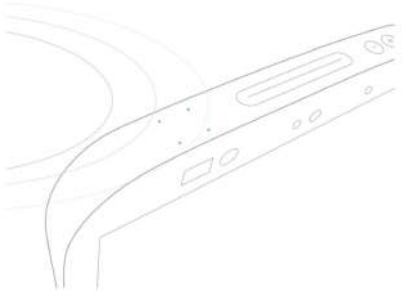


Figura 1.5: Posizione microfoni

- **scheda audio** : da 20 W composto da:
  - equalizzatore ad alta fedeltà;
  - due altoparlanti di fascia media;

- due *tweeter*;
- radiatori passivi;

Tutti ricoperti da tessuto acusticamente trasparente ed infine sulla base del robot si ha

- un *subwoofer*[2]

In sintesi, il sistema audio con queste specifiche offre un suono di alta qualità, con una potenza di 20W, un'elaborazione precisa delle frequenze grazie all'equalizzatore, un suono avvolgente grazie al subwoofer, una riproduzione nitida delle frequenze medie grazie agli altoparlanti di fascia media e delle alte frequenze dettagliate grazie ai tweeters. I radiatori passivi, invece, contribuiscono a migliorare la resa dei bassi senza necessità di un amplificatore separato, e il tessuto acusticamente trasparente garantisce una buona diffusione del suono senza distorsioni. Questo insieme di componenti rende il sistema audio capace di fornire un'esperienza audio immersiva e di alta fedeltà.

Il tutto mostrato in Figura 1.6 :

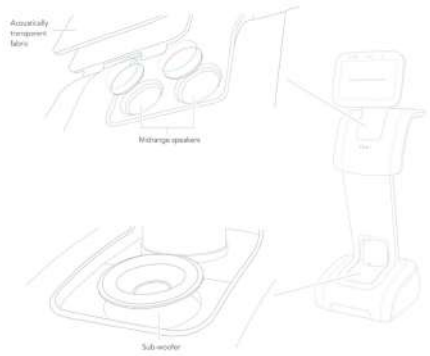


Figura 1.6: audio di Temi

[2] diffusore dedicato espressamente alle frequenze audio più basse

- **quattro fotocamere** (Figura 1.7):

  - ***HIGH RESOLUTION CAMERA***:

    - **13 MP**: La telecamera una risoluzione di 13 megapixel il che significa che può misurare la profondità degli oggetti e aggiornare la mappa tridimensionale 30 volte al secondo.

    - **Autofocus**: La fotocamera è dotata di un sistema di *autofocus*, che permette di regolare automaticamente la messa a fuoco dell'immagine per garantire che gli oggetti siano nitidi e ben definiti.

    - **1080p@30FPS**: La fotocamera può registrare video a una risoluzione di 1080p cioè 1920x1080 pixel (*Full* HD) e a una frequenza di 30 fotogrammi al secondo. Questo fornisce un'alta qualità video e una riproduzione fluida.

    - **FOV 75 gradi**: FOV sta per "*Field of View*" (campo visivo) e indica l'ampiezza angolare dell'area coperta dalla fotocamera. Un angolo FOV di 75 gradi significa che la fotocamera è in grado di catturare un'ampia area nell'inquadratura.

    - **Digital zoom**: La fotocamera supporta il digital zoom, che consente di avvicinare o allontanare l'inquadratura digitalmente, se necessario.

    - **5-Element lens**: La fotocamera è dotata di una lente a 5 elementi, che aiuta a migliorare la qualità delle immagini catturate.

- **Backside illumination sensor**: Il sensore della fotocamera è di tipo "backside illumination" (BSI), che migliora la sensibilità alla luce e contribuisce a ottenere immagini più luminose e dettagliate anche in condizioni di scarsa illuminazione.

- ***Hybrid IR filter***: La fotocamera utilizza un filtro infrarosso ibrido, che aiuta a ridurre l'effetto di interferenza e migliora la precisione e la fedeltà dei colori nell'immagine.

- Due ***WIDE CAMERA*** una fotocamera grandangolare progettata specificamente per la navigazione remota e la cattura di ampi angoli di visione. Le specifiche riportate per questa fotocamera sono le seguenti:

  - **13 MP**: La fotocamera ha una risoluzione di 13 megapixel, il che significa che può catturare immagini con una notevole quantità di dettagli e definizione anche in un'ampia area di vista.

  - **FOV 110 gradi**: FOV sta per "*Field of View*" (campo visivo) e indica l'ampiezza angolare dell'area coperta dalla fotocamera. Un angolo FOV di 110 gradi significa che la fotocamera è in grado di catturare un'ampia area nell'inquadratura, permettendo di ottenere una visione più estesa dell'ambiente circostante.

  - **1080p@30FPS**: La fotocamera può registrare video a una risoluzione di 1920x1080 pixel (*Full HD*) e a una frequenza di 30 fotogrammi al secondo. Questo offre una qualità video elevata e una riproduzione

- fluida, ideale per la navigazione remota e la registrazione di video ad alta qualità.

- **Digital zoom**: La fotocamera supporta il *digital zoom*, che consente di avvicinare o allontanare l'inquadratura digitalmente, se necessario, anche con un'ampia visuale.

- ***TOF DEPTH CAMERA*** basata sulla tecnologia *Time-of-Flight* (TOF) che viene utilizzata per misurare la profondità degli oggetti e rilevare la distanza tra la telecamera e gli oggetti circostanti. Questo tipo di telecamera emette impulsi di luce infrarossa verso gli oggetti circostanti e misura il tempo impiegato per il ritorno dei riflessi di luce. Utilizzando queste informazioni di tempo, è possibile calcolare la distanza degli oggetti e creare una mappa tridimensionale dello spazio circostante.Le specifiche riportate per la "*TOF depth camera*" sono le seguenti:

  - **30 FPS**: La telecamera è in grado di acquisire e fornire dati a una frequenza di 30 fotogrammi al secondo. Questo significa che può misurare la profondità degli oggetti e aggiornare la mappa tridimensionale 30 volte al secondo.

  - **Fino a 5 metri**: La telecamera è in grado di rilevare la profondità degli oggetti fino a una distanza massima di 5 metri dalla telecamera stessa. Ciò significa che può misurare la distanza degli oggetti nell'intervallo da vicino fino a 5 metri di distanza.

- **70 FOV**: FOV sta per "*Field of View*" (campo visivo) e indica l'ampiezza angolare dell'area che la telecamera è in grado di coprire. Un angolo FOV di 70 gradi significa che la telecamera è in grado di coprire un'area di circa 70 gradi di larghezza.

Queste specifiche rendono la "TOF depth camera" adatta per una varietà di applicazioni, come il rilevamento degli oggetti, il tracciamento dei movimenti, la mappatura di ambienti e il riconoscimento di gesti, tra le altre cose. La capacità di acquisire dati di profondità in tempo reale e la sua lunga distanza operativa la rendono utile in contesti come la robotica, la visione artificiale e altri settori in cui la comprensione della profondità dello spazio è essenziale.

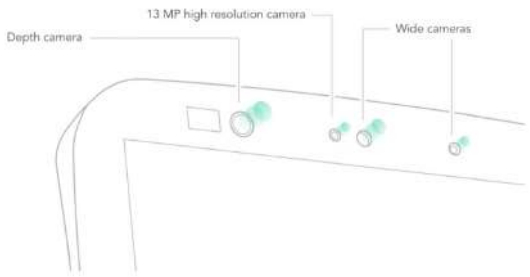



Figura 1.7: Fotocamere di Temi

- **due CPU** (Figura 1.8):

  - **CPU 1** che si divide in due parti:

    - **Dual-core Cortex-A72 1.8GHz**: Questa parte della CPU è composta da due core basati sull'architettura ARM Cortex-A72, con una frequenza di clock di 1.8GHz. Gli ARM Cortex-A72 sono core ad alte prestazioni,

progettati per gestire compiti intensivi e richiedenti calcoli più complessi. Essendo dual-core, questa parte della CPU è in grado di offrire una maggiore potenza di calcolo rispetto a un singolo core.

- **Quad-core Cortex-A53 1.4GHz**: Questa parte della CPU è composta da quattro core basati sull'architettura ARM Cortex-A53, con una frequenza di clock di 1.4GHz. Gli ARM Cortex-A53 sono core ad alta efficienza energetica, progettati per compiti meno intensivi e per ridurre il consumo di energia. Essendo quad-core, questa parte della CPU offre un bilanciamento tra potenza di calcolo e risparmio energetico.

Questo tipo di configurazione a *big.LITTLE*, con una combinazione di core ad alte prestazioni e core ad alta efficienza energetica, è progettato per bilanciare le prestazioni e l'efficienza del robot in base alle esigenze dell'utente. I core Cortex-A72 sono attivati per attività più intensive, mentre i core Cortex-A53 sono utilizzati per attività meno impegnative, riducendo il consumo di energia quando non è richiesta tutta la potenza di calcolo. Inoltre il sistema operativo del dispositivo è basato sull'architettura di Android 11 e include un launcher e un'interfaccia utente personalizzati per migliorare l'interazione con il dispositivo.

Questa combinazione di CPU e sistema operativo offre un buon equilibrio tra prestazioni, efficienza e personalizzazione dell'esperienza utente.

- **CPU 2** che si divide in due parti:

  - **Dual-core Cortex-A72 1.8GHz**: Questa parte della CPU è simile alla CPU 1 ed è composta da due core basati sull'architettura ARM Cortex-

A72, con una frequenza di clock di 1.8GHz. Essendo dual-core e basati su Cortex-A72, questi core offrono prestazioni elevate, ideali per compiti che richiedono maggiore potenza di calcolo.

- **Quad-core Cortex-A53 1.4GHz**: Anche in questa parte della CPU, ci sono quattro core basati sull'architettura ARM Cortex-A53, con una frequenza di clock di 1.4GHz. I core Cortex-A53 sono progettati per l'efficienza energetica, consentendo di gestire attività meno impegnative con un consumo ridotto di energia.

Questa specifica indica che il dispositivo utilizza una configurazione simile alla *big.LITTLE architecture*, ma con due CPU separate, ognuna composta da un dual-core Cortex-A72 e un quad-core Cortex-A53. Questo approccio permette al dispositivo di ottenere una maggiore potenza di calcolo quando necessario, utilizzando i core Cortex-A72, e di risparmiare energia durante compiti meno intensivi sfruttando i core Cortex-A53.

La differenza tra le due è:

- CPU 1 è progettata per l'interazione umana con il robot. Funge da "*Human Interaction Computer*" e gestisce le attività legate all'interfaccia utente e all'interazione dell'utente con il dispositivo. Questa CPU è supportata dall'operating system Android 11, che è progettato per fornire un'esperienza utente avanzata e aggiornata.

- CPU 2 è il "*Main Navigation Computer*" del robot ed è basata sull'architettura ARM. È progettata per gestire il sistema di navigazione principale del

robot. Questa CPU è ottimizzata per fornire elevate prestazioni nel campo della navigazione, consentendo al robot di offrire funzioni avanzate di navigazione, come la guida autonoma, con un consumo energetico ridotto.

Il risultato di questa configurazione è un dispositivo in grado di gestire un'ampia varietà di compiti, offrendo prestazioni elevate quando necessario e mantenendo un basso consumo energetico durante le attività quotidiane. Questo contribuisce a una migliore esperienza utente, con una risposta rapida dell'interfaccia e una maggiore autonomia della batteria, rendendo il dispositivo versatile e efficiente per l'uso quotidiano.




Figura 1.8 : CPU
presenti in temi

- il robot è dotato di **ricarica**, situata alla base (Figura 1.8):

  - **8 ore di funzionamento per carica**: il robot può funzionare in modo autonomo per un periodo di circa 8 ore dopo essere stato completamente ricaricato.

- **L'autonomia della batteria** : è importante poiché determina per quanto tempo il robot può essere utilizzato prima di dover essere ricaricato.

- **Autonomo di ricarica - 220V/110V**: Questo significa che il robot può essere ricaricato in modo autonomo collegandolo a una presa di corrente domestica standard di 220V o 110V. Questo lo rende compatibile con diverse tensioni di alimentazione utilizzate in diversi paesi.

- ***High-performance docking station***: La *docking station* (stazione di ricarica) è progettata per fornire una ricarica veloce ed efficiente al dispositivo.

- **Cellule di litio-ion**: il robot è dotato di celle al litio-ion che sono un tipo di batterie ricaricabili comunemente utilizzate nei dispositivi elettronici portatili. Le batterie al litio-ion sono apprezzate per la loro alta densità energetica, lunga durata e capacità di mantenere una carica stabile nel tempo.

Queste caratteristiche combinano una lunga autonomia operativa con una comoda ricarica, rendendo il dispositivo più pratico e versatile per l'uso quotidiano.

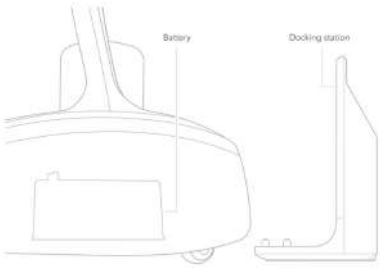



Figura 1.8: Batteria di Temi

- **Mobilità** : il robot è dotato di due motori *brushless* indipendenti ad alta efficienza ed da degli *encoder* magnetici ad alta risoluzione per garantire un movimento fluido e preciso. La capacità di sterzata a raggio zero rende il robot altamente manovrabile, mentre la velocità massima di 1 metro al secondo offre un'efficace copertura delle distanze. I pneumatici personalizzati sono progettati per migliorare la presa, la stabilità e la gestione delle vibrazioni, contribuendo a una guida ottimale in diverse situazioni e ambienti. Queste caratteristiche combinate forniscono un sistema di movimento avanzato e affidabile per il robot, garantendo un'esperienza di guida sicura e piacevole.

In dettaglio è possibile individuare quattro aspetti legati al movimento di Temi (Figura 1.9):

1. **Pneumatici personalizzati**: nel robot sono presenti dei pneumatici adattati per ottenere una presa ottimale, stabilità e gestione delle vibrazioni: I pneumatici personalizzati del robot sono progettati per garantire un'ottima presa su diverse superfici, migliorare la stabilità del robot durante il movimento e ridurre le vibrazioni per una guida confortevole. Questo contribuisce a una maggiore aderenza e sicurezza del robot mentre si sposta.

2. **Sistema di Propulsione:** Due motori brushless a corrente continua (*Brushless DC motors*) indipendenti da 50 W ciascuno: I motori brushless sono dispositivi elettrici che offrono una propulsione efficiente e priva di spazzole, garantendo maggiore durata e minore manutenzione rispetto ai motori tradizionali con spazzole. Il fatto che siano due motori indipendenti permette un maggiore controllo del movimento e della manovrabilità del robot.

3. ***Encoder*** **magnetici ad alta risoluzione**: Gli encoder magnetici sono sensori che consentono di misurare con precisione la posizione e la velocità dei motori. Gli encoder ad alta risoluzione assicurano un movimento fluido e una guida precisa, migliorando la stabilità del robot e la gestione delle velocità.

4. **Manovrabilità**: Raggio di sterzata zero: Questa caratteristica indica che il robot è in grado di ruotare su se stesso senza spazio di sterzata, permettendo manovre agili e precise. Questa capacità è particolarmente utile in spazi ristretti o quando è necessario cambiare rapidamente direzione.

5. **Velocità** : Fino a 1 metro al secondo: Il robot è in grado di raggiungere una velocità massima di 1 metro al secondo, il che corrisponde a circa 3,6 km/h. Questa velocità offre un equilibrio tra efficienza del movimento e sicurezza, consentendo di coprire distanze con una velocità adeguata per molte applicazioni.

6. **Pneumatici personalizzati** : Pneumatici adattati per ottenere una presa ottimale, stabilità e gestione delle vibrazioni: I pneumatici personalizzati del robot sono progettati per garantire un'ottima presa su diverse superfici, migliorare la stabilità del robot durante il movimento e ridurre le vibrazioni per una guida confortevole. Questo contribuisce a una maggiore aderenza e sicurezza del robot mentre si sposta.




Figura 1.9: Batteria di Temi

- **Vassoio** (Figura 1.10) : Il vassoio ergonomico contiene un caricatore wireless standard per il un telefono/cellulare. Questo permette di caricare comodamente il telefono/cellulare senza fili mentre si utilizza il robot temi. Nel caso di temi V3, il vassoio è dotato di una presa di alimentazione da 12V DC e di 4 fori filettati per l'aggiunta di accessori. Questa caratteristica offre la possibilità di personalizzare il vassoio con accessori aggiuntivi o dispositivi che possono essere alimentati tramite la presa di alimentazione da 12V DC. I 4 fori filettati consentono di fissare in modo sicuro gli accessori compatibili. Le dimensioni del vassoio sono di circa 30 cm x 25 cm (0.98 piedi x 0.8 piedi), offrendo uno spazio sufficiente per posizionare oggetti o dispositivi su di esso.Per quanto riguarda il carico massimo del vassoio, può sopportare fino a 6.6 libbre o 3 kg, ma ciò dipende dalla configurazione specifica dell'assemblaggio.



Figura 1.10: Vassoio in Temi

### 2.1.3 Sistema di navigazione ROBOX™

Il sistema di navigazione *ROBOX*™ su Temi robot offre un'interazione e una navigazione autonoma tra umani e robot. Temi è dotato di molteplici sensori e funzionalità avanzate per garantire una navigazione precisa e sicura e supporta:

- ***2D Mapping*** : Temi è in grado di creare una mappa bidimensionale dell'ambiente circostante, di comprendere la disposizione degli oggetti e degli ostacoli.

- ***3D Localization*** : Utilizzando sensori appositi, Temi è in grado di localizzarsi nello spazio tridimensionale con alta precisione, consentendo di muoversi in modo accurato e affidabile.

- **Navigation** : Temi è dotato di capacità di navigazione autonoma, che gli permettono di spostarsi autonomamente all'interno di un ambiente e raggiungere diverse posizioni specificate.

- **Rilevamento e Tracciamento dell'Utente** : Temi è in grado di rilevare la presenza degli utenti nell'ambiente e seguirli nel loro movimento, permettendo una interazione dinamica tra il robot e le persone.

- **Evitamento Ostacoli** : Grazie ai sensori, Temi può rilevare gli ostacoli presenti nel suo percorso e evitare collisioni, assicurando una navigazione sicura.

- **Pianificazione del Percorso** : Temi può pianificare il suo percorso in modo intelligente, considerando gli ostacoli e le condizioni dell'ambiente, per raggiungere la destinazione in modo efficiente.

- **Navigazione Completamente Autonoma** : Temi è in grado di muoversi autonomamente senza bisogno di dispositivi esterni per la navigazione e il tracciamento degli utenti. Questo consente al robot di essere completamente autonomo durante l'interazione con l'ambiente e le persone.

- **Navigazione in Tempo Reale** : Grazie all'analisi in tempo reale dei dati provenienti dai sensori e alla fusione di informazioni, Temi può prendere decisioni rapide e sicure per una navigazione accurata e sicura.

Il sistema di navigazione *ROBOX*™ su Temi è progettato per fornire un'esperienza di navigazione autonoma, interattiva e sicura. Grazie a una varietà di sensori e alla potente *CPU*, il robot è in grado di creare mappe, rilevare oggetti e utenti, pianificare percorsi e muoversi in modo preciso, offrendo un'esperienza intuitiva ed efficiente per l'utente.

Precisamente il sistema di navigazione *ROBOX*™ presente nel robot Temi utilizza la tecnica SLAM (*Simultaneous Localization and Mapping*) per fornire una navigazione avanzata e autonoma.

SLAM è una tecnica di intelligenza artificiale utilizzata dai robot e dai sistemi autonomi per creare una mappa dell'ambiente circostante e localizzarsi in essa simultaneamente. Il processo SLAM coinvolge il rilevamento degli ostacoli e delle caratteristiche dell'ambiente attraverso l'utilizzo di sensori come il LIDAR (*Light Detection and Ranging*), telecamere, sensori di profondità, e altre unità di misurazione.

Quando il robot si muove all'interno di un ambiente, i sensori rilevano informazioni sull'ambiente circostante e le utilizzano per costruire una mappa tridimensionale in tempo reale. Contestualmente, SLAM calcola la posizione e l'orientamento del robot all'interno di questa mappa in continuo aggiornamento.

Quindi, grazie alla tecnica SLAM, il robot temi è in grado di navigare autonomamente, evitare ostacoli e seguire le istruzioni degli utenti mentre si sposta in modo preciso e sicuro. Questo sistema di navigazione avanzato permette al robot di essere completamente autonomo senza bisogno di mappe preesistenti o di dispositivi esterni per la navigazione.

In sintesi, il sistema di navigazione ROBOX™ basato sulla tecnica SLAM è uno degli elementi chiave che rendono il robot Temi un compagno utile e interattivo, in grado di esplorare autonomamente gli ambienti, creare mappe dettagliate e fornire una navigazione sicura ed efficiente all'interno delle abitazioni e degli ambienti sociali (Figura 1.11).

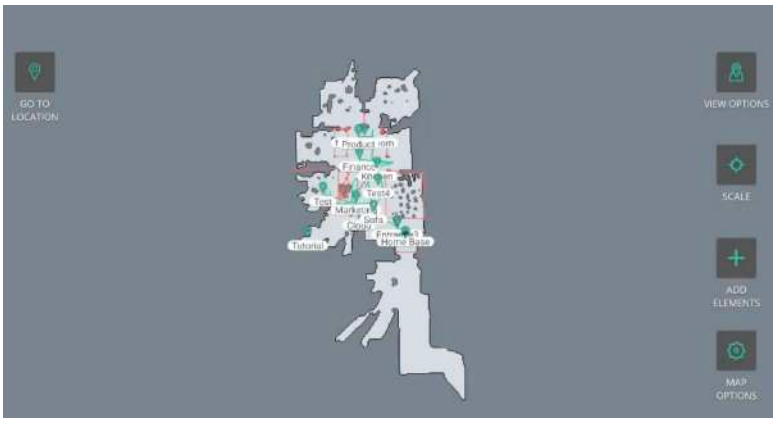



Figura 1.11: mappa creata con tecnica SLAM in Temi Robot [4]

### 2.1.4 Temi OS

TEMI OS è un sistema operativo estremamente intuitivo che equipaggia il robot temi, consentendogli di "venire", "sentire", "parlare", "vedere" e "capire", imparando costantemente come servire al meglio.

Il sistema operativo Temi permette di gestire in modo centralizzato e personalizzato tramite il "*Temi Center*" - un sistema di gestione online. Attraverso questo centro, è possibile controllare e personalizzare le funzioni del robot temi per soddisfare le esigenze specifiche degli utenti.

Ecco ulteriori dettagli sui punti chiave del TEMI OS e le funzionalità che offre:

1. **Integrazione AI** : TEMI OS integra i migliori motori di intelligenza artificiale disponibili per offrire un'esperienza di interazione avanzata. Ciò consente a Temi di connettersi ai servizi preferiti e di imparare continuamente dai dati e dall'ambiente circostante. La sua capacità di autoapprendimento gli permette di ottimizzare le risposte e le azioni per soddisfare le esigenze specifiche degli utenti.

2. ***Voice Interaction*** : Temi è dotato di un sofisticato sistema di interazione vocale. Grazie all'utilizzo di tecnologie di Auto *Speech Recognition* (ASR), *far field voice technology*, *natural language processing* (NLP), *speech-to-text* e *text-to-speech* (TTS), temi è in grado di ascoltare, identificare, comprendere e rispondere alla voce dell'utente. Questo rende l'interazione con il robot temi naturale e senza bisogno di comandi complessi.

3. ***User Interface*** : La user interface di Temi è stata progettata in modo intelligente per rendere l'esperienza dell'utente coinvolgente e intuitiva. Il *display* del robot Temi offre informazioni visive e risposte visive alle interazioni vocali, rendendo l'esperienza di utilizzo più interattiva e interessante.

4. ***Navigation System*** : Il sistema di navigazione di temi è altamente avanzato e si basa sulla tecnica SLAM (*Simultaneous Localization and Mapping*). Questa tecnologia consente al robot di creare una mappa dell'ambiente circostante in tempo reale, localizzarsi all'interno della mappa e navigare in modo autonomo evitando ostacoli. Temi è in grado di adattarsi a eventuali cambiamenti nell'ambiente e ottimizzare i percorsi per raggiungere le destinazioni desiderate.

5. ***Facial Recognition*** : Il sistema di riconoscimento facciale supportato da temi gli permette di identificare le persone con cui interagisce. Questa funzionalità è utile per personalizzare l'esperienza dell'utente, ad esempio accogliendo gli utenti con il loro nome o attivando preferenze personalizzate.

6. ***Open to 3rd Party Apps*** : TEMI OS è aperto e supporta l'integrazione di applicazioni di terze parti tramite un SDK. Ciò significa che gli sviluppatori possono creare applicazioni personalizzate per Temi, aggiungendo nuove funzionalità e servizi in base alle esigenze degli utenti.

7. ***Security*** : TEMI OS implementa rigorose misure di sicurezza e protocolli avanzati per proteggere le informazioni e garantire un'esperienza utente sicura. La sicurezza è rafforzata ulteriormente dalla tecnologia di sicurezza IoT Firedome.

## 2.2 Ambiente Android

Android è un sistema operativo che negli ultimi anni sta animando buona parte dei dispositivi mobili e non, tanto da avere una presenza costante nella vita quotidiana, offrendo una ricca struttura per costruire applicazioni e giochi in un ambiente di linguaggio Java.

La società *Android.inc* è stata fondata nell'ottobre del 2003 da *Andy Rubin* e il suo sistema operativo ha avuto una veloce diffusione per diversi motivi, uno dei più importanti, oltre ad essere leggero e robusto, è attribuito al fatto di essere un software open source, infatti è basato su un *kernel Linux*, questo ha attratto l'interesse di tanti sviluppatori.

La prima release di Android è uscita in versione beta nel novembre 2007 con il rilascio delle SDK (*Software Development Kit*). L'SDK era composto dagli strumenti di sviluppo, la documentazione in inglese, le librerie, un emulatore del dispositivo, tutorial ed esempi di progetti.

Android 1.0, la prima versione commerciale, è stata rilasciata nel 2008 e comprendeva lo store, il *browser*, il supporto alle reti *wi-fi* e della fotocamera, i servizi di posta elettronica, la gestione del file system (creazione, cancellazione e ridenominazione) e le prime *Google App*.

Dall'aprile del 2009, le versioni di Android, dalla 1.5 in poi, hanno cominciato a prendere il nome di prodotti dolciari e procedono in ordine alfabetico.

Le applicazioni Android, come già anticipato, sono sviluppate in Java e gli strumenti dell'*SDK* servono per compilare il codice e produrre un file APK (*Android Package*), che grazie ad esso l'applicazione potrà essere installata sul dispositivo e vivere all'interno di un ambiente di sicurezza che prende il nome di "*sandbox*". Nel S.O. Android ogni singola applicazione è rappresentata da un processo distinto dagli altri e gira in maniera indipendente da tutte le altre applicazioni. I processi vengono creati quando uno o più componenti dell'applicazione sono eseguiti e vengono terminati quando il S.O. non ne necessita più. Questa coordinazione dei processi permette di gestire la *RAM (random access memory)* in modo intelligente e di liberarla per consentire l'esecuzione di nuovi processi.

Android attua il principio del minimo privilegio, cioè, ogni applicazione può accedere ad un numero limitato di risorse che gli permettono di svolgere il proprio lavoro e non di più, questo implica maggiore sicurezza poiché ai processi applicativi viene impedito di accedere alle parti vitali e delicate del sistema operativo, il cui danneggiamento potrebbe comprometterne in modo irreversibile il funzionamento.

Inoltre esistono metodi per condividere dati tra applicazioni e per utilizzare i servizi di sistema, per esempio si inizializzano due applicazioni con lo stesso *LinuxID* e si eseguono nello stesso processo, queste condivideranno la stessa "*sendbox*" (solo se le due applicazioni sono state firmate con lo stesso certificato).

Le Applicazioni possono accedere a diverse risorse come il Bluetooth, la fotocamera, i contatti utente, gli *SMS*, i supporti rimovibili e altro, però per poterlo fare devono ottenere delle autorizzazioni che, fino alla versione precedente alla 6.0 del *S.O Android*, dovevano essere approvate in fase di installazione, ma dalla 6.0 l'approvazione dell'accesso viene effettuata in fase di esecuzione dell'applicazione.

## 2.2.1 Android Studio

Per questa tesi è stato usato *Android Studio* IDE[3] (*Integrated Development Environment*), cioè un ambiente di sviluppo integrato per gli sviluppatori Android. È stato ideato da Google e la prima versione *stable* risale al dicembre 2014.

Questo IDE è basato sul software di *JetBrains IntelliJ* IDEA ed è disponibile per le piattaforme *Windows*, *Mac OS X* e *Linux* sostituendo *Eclipse* come IDE principale per lo sviluppo di app native.

Riguardo al *download*, seguendo il *link*[4], è possibile trovare l'ultima versione disponibile.




Figura 1.18: Android studio: logo

[3] https://developer.android.com/studio

[4] http://developer.android.com/ sdk/index.html

Android Studio fornisce una vasta gamma di funzionalità e risorse per semplificare il processo di sviluppo di app per dispositivi Android.

Le caratteristiche principali si possono classificare in:

1. **Editor di codice avanzato**: Android Studio offre un editor di codice avanzato che supporta la scrittura di codice Java e Kotlin, i due principali linguaggi di programmazione utilizzati per lo sviluppo Android. L'editor offre suggerimenti intelligenti, evidenziazione della sintassi, refactoring del codice e integrazione con Git per la gestione delle versioni del codice.

2. **Emulatore Android integrato**: Android Studio include un emulatore Android virtuale che consente agli sviluppatori di testare le proprie app su diverse configurazioni di dispositivi Android senza dover possedere fisicamente ciascun dispositivo. Questo è estremamente utile per il debug e il test delle app su varie dimensioni dello schermo e versioni di Android.

3. **Designer di layout visuale**: Android Studio offre un designer di layout visuale che semplifica la creazione dell'interfaccia utente delle app Android. Gli sviluppatori possono trascinare e rilasciare elementi dell'interfaccia utente per creare layout complessi e vedere immediatamente come appariranno sul dispositivo.

4. **Strumenti di analisi delle prestazioni**: L'IDE include strumenti avanzati per l'analisi delle prestazioni delle app Android. Gli sviluppatori possono monitorare l'utilizzo della CPU, della memoria e della rete delle proprie app per identificare e risolvere eventuali problemi di prestazioni.

5. **Integrazione con Google Services**: Android Studio è strettamente integrato con i servizi Google, consentendo agli sviluppatori di sfruttare funzionalità come l'autenticazione utente tramite account Google, mappe, notifiche push e molto altro ancora.

6. **Supporto per librerie e plugin**: Android Studio supporta l'uso di librerie di terze parti e plugin che estendono le sue funzionalità. Ciò consente agli sviluppatori di integrare facilmente funzionalità aggiuntive nelle proprie app.

7. **Build automatizzati**: Android Studio automatizza il processo di compilazione delle app e la gestione delle dipendenze. Gli sviluppatori possono generare facilmente file APK pronti per la distribuzione.

8. **Integrazione con Firebase**: Firebase è una piattaforma di sviluppo mobile di Google che offre una vasta gamma di servizi cloud per migliorare le app Android. Android Studio è integrato con Firebase, semplificando l'uso di servizi come l'autenticazione, l'analisi, le notifiche push e altro ancora.

Per lo sviluppo del progetto è stata usata la versione "*Android studio Electric Eel*" realizzata il 21 gennaio 2023 in quanto il tablet di *Temi V3* funziona su *Android 11* mentre in merito ai linguaggi adoperati per lo sviluppo la scelta è ricaduta su *Java*[5] (versione 8).

[5] *Linguaggio Object Oriented*

### 2.2.2 Android Debug Bridge per la connessione a Temi

L'Android Debug Bridge[6] è un tool di linea di comando versatile che permette di comunicare con un'istanza dell'emulatore o con un device Android supportato. È un programma di tipo *client-server* che include tre componenti:

- Un client che viene eseguito sulla macchina che viene utilizzata per sviluppare il software e può essere invocato dal terminale con una linea di comando.
- Un server che viene eseguito come processo di background sull'emulatore o sul device usato per i test.
- Un demone che esegue come un processo di background su ogni istanza di emulatore o device.

ADB permette delle operazioni molto comode per lo sviluppo di applicazioni. Tramite esso è infatti possibile avviare un terminale sul *device* o macchina virtuale usata per i test, inviando ad essa tutti i comandi di una normale *shell*[7] di tipo *bash*[8]. È anche possibile utilizzarla al fine di copiare file dalla macchina virtuale a quella reale e viceversa, ovviamente è possibile utilizzare le stesse potenzialità anche per i device.

ADB è stata usata, nello sviluppo, per eseguire l'applicazione prodotta in android studio sul robot Temi in quanto poiché non esiste un accesso *USB* si comunica con il robot Temi tramite *Android Debug Bridge* (ADB).

---

[6] https://developer.android.com/tools/adb

[7] https://it.wikipedia.org/wiki/Shell_(informatica)

[8] https://it.wikipedia.org/wiki/Bash

I passaggi da intraprendere per eseguire un'applicazione su robot Temi, sono i seguenti:

1. Si scarica adb (Android Debug Bridge) sul computer su cui è sviluppata l'app da eseguire su temi

2. Si fa in modo che il computer e Temi siano sulla stessa rete WiFi.

3. Si scopre l'indirizzo IP privato del Temi tramite [Settings] > [Temi Developer Tools] > [ADB Connection to Temi] >[Open Port] (sul robot Temi)

4. Ci si connette a Temi con adb da terminale tramite il comando "adb connect [IP address privato del robot] "

5. Su android si seleziona il Device che si presenta come (*aka Rockchip*) e si esegue applicazione. Al caricamento l'applicazione viene aperta sul robot.

### 2.2.3 Room Database

Room[9] è una libreria di persistenza di database per Android che adotta un approccio *ORM* (Object Relational Mapping) per semplificare l'interazione tra oggetti *Java/Kotlin* e il *database SQLite*.

Nell'approccio *ORM*, gli oggetti *Java/Kotlin* sono mappati direttamente alle tabelle del database e invece di scrivere manualmente codice *SQL* per creare tabelle, inserire dati o recuperare risultati, Room si occupa di gran parte di questo lavoro in modo automatico,

[9] https://developer.android.com/jetpack/androidx/releases/room

riducendo il codice *boilerplate* (marcato) e semplificando notevolmente lo sviluppo dell'applicazione.

L'architettura tipica di un progetto che fa uso di libreria *Room* è la seguente (Figura 1.21):

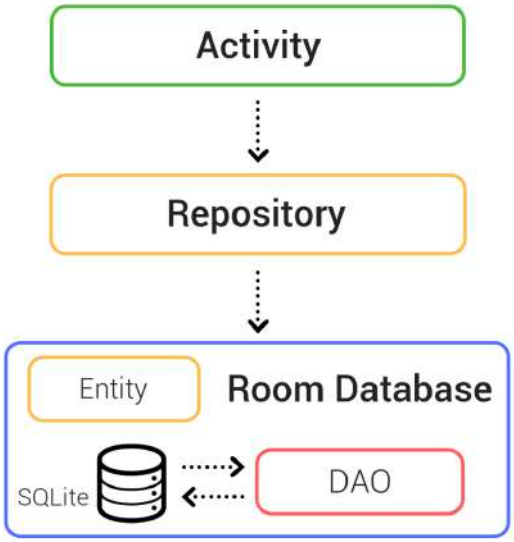



Figura 1.21 : architettura progetto con Room
[5]

Dove:

- ***Activity*** : frame in cui risiede il codice per interagire con il database.

- ***Repository*** : classe che astrae l'accesso a più origini dati. Il *repository* non fa parte delle librerie *Architecture Components*, ma è una best practice consigliata per la separazione del codice e l'architettura. Una classe *Repository* gestisce le operazioni sui dati e fornisce un'API pulita al resto dell'app per i dati.

- ***Room Database*** : composto da tre classi :

    1. ***Entity***, definisce le righe del database SQLite. Questo sarà essenzialmente un modello dei dati che desideri archiviare.

    2. **DAO** (oggetto di accesso ai dati), contiene le query SQL e le associazioni alle chiamate di metodo. Il compilatore controlla l'SQL e genera query da annotazioni di convenienza per query comuni, come *@Insert, @Update, @Delete e @Query*.

    3. ***Database*** (la banca dati ): Questa classe contiene il codice necessario per creare un'istanza dell'oggetto database *SQLite*. In genere, è necessaria solo un'istanza del database *Room* per l'intera *app*. Ecco perché spesso viene utilizzato il pattern *singleton* in tale parte.

## 2.3 Python

*Python*[10] è un linguaggio di programmazione ad alto livello adatto a sviluppare applicazioni distribuite, scripting, computazione numerica e testing, inoltre è interpretato e general-purpose. È un linguaggio multi-paradigma in quanto supporta la programmazione Object-Oriented ma anche quella funzionale. Ha tre principali obiettivi: semplicità, dinamicità e flessibilità.

Python utilizza variabili non tipizzate e ideazione per la sintassi delle specifiche (invece delle classiche parentesi graffe). Il controllo dei tipi è forte e avviene a runtime: una

[10] https://www.python.org/

variabile è un contenitore a cui viene associata un'etichetta che può essere a sua volta associata a diverse variabili anche di tipo diverso, durante il suo tempo di vita.

Python utilizza un Garbage Collector, quindi non è possibile gestire la memoria dinamica a proprio piacimento.

Infime Python è un linguaggi interpretato seppur utilizza alcuni sistemi per velocizzare l'interpretazione del codice sorgente, senza dover reinterpretare codice già interpretato (attraverso l'utilizzo di bytecode).

Python è stato utilizzato per la registrazione e il riconoscimento del volto, come vedremo in un paragrafo successivo:

### 2.3.1 Libreria : face_recognition

La libreria *face_recognition*[11] è una libreria *Python* open-source usata nello sviluppo del progetto per che facilita il riconoscimento dei volti nelle immagini. Questa libreria è basata su modelli di machine learning pre-addestrati ed è stata sviluppata per semplificare il processo di riconoscimento dei volti utilizzando *Python*.

Il suo funzionamento si può riassumere in quattro fasi:

1. **Rilevamento dei volti**: La libreria utilizza un modello di machine learning pre-addestrato per individuare i volti presenti in un'immagine.

[11] https://pypi.org/project/face-recognition/

2. **Estrazione delle caratteristiche facciali**: Una volta individuati i volti, la libreria estrae le caratteristiche facciali di ciascun volto.

3. **Confronto delle caratteristiche**: Quando si vuole riconoscere un volto specifico, ad esempio confrontandolo con un database di volti noti, la libreria calcola le caratteristiche facciali del volto da identificare e le confronta con le caratteristiche facciali dei volti noti.

4. **Riconoscimento**: In base al confronto delle caratteristiche, la libreria può stabilire se il volto sconosciuto corrisponde a uno dei volti noti nel database.

Nello sviluppo la registrazione del volto dell'utente è stata realizzata facendo in modo che l'utente acceda alla fotocamera del dispositivo e cattura l'immagine del suo volto, questa viene inviata al server Python. Quando il server Python riceve l'immagine verifica la presenza di volto nella foto e nel caso restituisce un feedback negativo altrimenti genera in totale autonomia la chiave biometrica corrispondente e la invia al codice Android Studio che insieme ad altre informazioni dell'utente le memorizza nel database dell'applicazione

Mentre il riconoscimento è stato realizzato facendo in modo che l'utente cattura l'immagine, che viene inviata al server Python. Quando il server Python riceve l'immagine verifica la presenza di volto nella foto e nel caso restituisce un feedback negativo altrimenti genera in totale autonomia la chiave biometrica corrispondente e la invia al codice Android Studio. A tal punto andiamo a confrontare questa nuova chiave biometrica con quelle salvate nel database e se viene trovata una corrispondenza, facendo uso del concetto di similarità, viene notifica l'attività all'utente.

# 3. Approccio proposto

In questo capitolo tratteremo la progettazione e l'implementazione dell'applicazione realizzata per il robot, si dettaglierà: il comportamento del robot in particolare come il robot interagisce con l'ambiente circostante; con gli utenti e come esegue le attività assegnate. Saranno, inoltre, dettagliate le interfacce dell'applicazione e il relativo funzionamento.

## 3.1 Analisi del comportamento del robot

### 3.1.1 Interazione con l'Utente

Il comportamento del robot nei confronti degli utenti è uno degli aspetti più rilevanti. Il robot Temi è progettato per essere interattivo ed efficiente nella comunicazione. Esso, infatti, utilizza la sintesi vocale per comunicare con gli utenti.

Il flusso vocale del robot Temi è composto da quattro componenti principali:

- *wakeup* : è la fase iniziale del flusso vocale, in cui il sistema si prepara a riconoscere un comando vocale. Di solito, il sistema è in uno stato di "riposo" e il "*wakeup*" è responsabile di attivare il riconoscimento vocale quando viene rilevata un'indicazione, come una parola chiave o un suono specifico

- ASR (*Automatic Speech Recognition*) : Questa componente è responsabile del riconoscimento vocale automatico. Durante questa fase, il sistema converte il flusso di suoni vocali in testo. In altre parole, traduce ciò che viene detto in parole scritte o in una rappresentazione testuale. L'ASR è fondamentale per capire cosa l'utente sta dicendo.

- NLP (*Natural Language Processing*) : Dopo che il testo è stato estratto tramite ASR, la fase successiva implica la comprensione del significato del testo. Questo può includere l'analisi della grammatica, la comprensione del contesto e l'estrazione di informazioni rilevanti dal testo. L'obiettivo è tradurre il testo in una forma che il sistema possa utilizzare per generare una risposta appropriata. Nei robot Temi, questa operazione può essere gestita attraverso l'uso di script e programmazione apposita per consentire al robot di comprendere e rispondere adeguatamente.

- TTS (*Text-to-Speech*) : La componente TTS è responsabile di prendere il testo generato dalla fase NLP e convertirlo nuovamente in una rappresentazione vocale. In altre parole, trasforma il testo in un flusso di suoni vocali in modo che il sistema possa rispondere all'utente in modo verbale.

Nell'applicazione, oggetto della presente Tesi, si è fatto uso della sintesi vocale per segnalare feedback positivi o negativi, da parte del robot all'utente, in risposta al successo o insuccesso delle diverse funzionalità implementate. Si è fatto uso nello specifico di :

- **feedback vocale** : tramite TTS si è fatto in modo che quando il robot Temi deve comunicare un feedback positivo o negativo all'utente, lo fa in forma vocale, quindi l'utente può ascoltarlo direttamente.

- **feedback visivo** : tramite Messaggi Testuali su Schermo il robot "Temi" mostra anche i feedback positivo o negativo all'utente sotto forma di messaggi testuali sullo schermo. Questa modalità è utile per garantire che l'utente possa leggere e confermare il feedback, specialmente in ambienti in cui il suono può essere disturbante o indesiderato.

La combinazione di feedback vocale e feedback visivo ha migliorato l'accessibilità e l'usabilità del sistema. I principali vantaggi che hanno portato a implementare entrambe le modalità sono legati a :

1. **Accessibilità** : L'uso combinato di feedback vocale e feedback visivo rende il sistema più accessibile a una varietà di utenti. Le persone con disabilità visive possono ascoltare i messaggi, mentre le persone con problemi uditivi possono beneficiare del feedback visivo.

2. **Chiarezza e Comprensione** : La combinazione di modalità di comunicazione diverse può migliorare la chiarezza e la comprensione delle informazioni fornite. Alcuni utenti potrebbero preferire la lettura dei messaggi testuali, mentre altri preferiscono l'ascolto del feedback vocale.

3. **Riduzione degli Errori** : Fornendo il feedback sia in forma vocale che testuale, si riduce la probabilità di errori di comprensione o fraintendimenti. Gli utenti possono verificare il feedback in entrambe le modalità per assicurarsi di averlo capito correttamente.

4. **Flessibilità Ambientale** : In ambienti in cui il rumore di fondo può essere elevato o in situazioni in cui il silenzio è richiesto, i messaggi testuali possono

essere particolarmente utili poiché non richiedono il suono. D'altra parte, in ambienti meno rumorosi o quando l'utente è lontano dallo schermo, il feedback vocale può essere prezioso.

5. **Risposta Rapida** : In situazioni in cui l'utente può dover rispondere rapidamente a un feedback, avere entrambe le modalità a disposizione può consentire una risposta più veloce e appropriata.

6. **Versatilità** : L'uso congiunto di feedback vocale e feedback visivo rende il sistema più versatile, adattabile a una gamma più ampia di situazioni e scenari di interazione.

In sintesi, l'implementazione di entrambi i tipi di feedback ha consentito una comunicazione più ricca e flessibile tra l'utente e il sistema, migliorando l'esperienza complessiva.

### 3.1.2 Navigazione e Movimento del robot

La navigazione e il movimento sono due aspetti fondamentali nella progettazione e nell'implementazione di robot. Questi aspetti sono cruciali per consentire al robot di spostarsi all'interno di un ambiente, interagire con gli oggetti e fornire servizi o informazioni agli utenti.

Nell’implementazione dell’applicazione per quanto riguarda navigazione e il movimento del robot Temi ci si è concentrati su tre aspetti :

- **Mappatura :** Il robot Temi crea e aggiorna una mappa dell'ambiente in cui si trova. Questo è essenziale per consentire al robot di comprendere la sua posizione e l'ambiente circostante. La precisione della mappatura è importante per una navigazione affidabile, e deve essere in grado di gestire le modifiche nell'ambiente, ad esempio spostamento di oggetti o cambiamenti nella disposizione degli oggetti

- **Pianificazione del Percorso :** Il robot deve pianificare un percorso ottimale per raggiungere una destinazione specifica all'interno dell'ambiente. L'efficienza nella scelta del percorso è importante per garantire che il robot raggiunga la sua destinazione nel minor tempo possibile. La gestione degli ostacoli è cruciale durante la pianificazione del percorso per evitare collisioni.

- **Evitamento Ostacoli :** Durante la navigazione, il robot deve essere in grado di rilevare gli ostacoli nel suo percorso e prendere decisioni in tempo reale per evitarli in modo sicuro. Questo richiede l'utilizzo di sensori e algoritmi di evitamento degli ostacoli.

Nell'implementazione del progetto si è fatto uso della mappa interna di temi e tramite metodi dell'SDK si è cercato di salvare, eliminare, aggiornare locazioni o punti d'interesse.

Per il percorso, invece, quando viene eseguito il robot Temi naviga lungo il percorso attraversando tutte le locazioni o i punti d'interesse presenti nel percorso creato. In particolare quando il robot Temi deve eseguire un percorso arriva alla posizione o al punto d'interesse e comunica all'utente tramite feedback visivo oppure vocale che è arrivato ad una certa posizione del percorso e poi continua la navigazione lungo il percorso. Quest'utilizzo è un aspetto importante per migliorare

l'interazione tra il robot e gli utenti, in quanto consente agli utenti di comprendere lo stato e il progresso del robot durante la navigazione.

### 3.1.3 Rilevamento dell'utente

Il rilevamento dell'utente è un aspetto fondamentale per i robot intelligenti e interattivi come il Temi Robot. Questo processo consente al robot di individuare la presenza di persone nell'ambiente circostante e reagire di conseguenza. Il rilevamento dell'utente può avvenire attraverso diverse modalità e sensori.

Nello sviluppo del progetto, della presente tesi, per far in modo che il robot all'attivare dall'attività, vadi alla posizione stabilità e attiva la funzione di rilevamento dell'utente sono stati utilizzati i sensori di rilevamento presenti nel robot Temi.
Dove l'idea implementata fa si che una volta rilevata la presenza di un utente, il robot può intraprendere diverse azioni come avviare il riconoscimento tramite volto dell'utente.

Nell'implementazione per tale operazioni sono stati utilizzati differenti metodi presenti nell'SDK di Temi che si rifanno al Detection Mode.

Il Detection Mode è una modalità operativa del robot Temi in cui il robot costantemente cerca la presenza di utenti nell'area circostante. Quando la Detection Mode è attivata, il robot terrà d'occhio l'ambiente circostante attraverso la telecamera RGB (fotocamera a colori) e cercherà di individuare la presenza di persone e se è presente più di un'utente, scegli quello più vicino e quello più centrale.

### 3.1.4 Riconoscimento dell'utente

Il riconoscimento dell'utente da parte di un robot è una funzionalità che consente al robot di identificare, tracciare e interagire con le persone o gli utenti che si trovano nelle vicinanze.

Nello sviluppo del progetto, della presente tesi, tale operazione è preceduta dalla fase di registrazione, infatti :

- Gli utenti prima registrano le loro informazioni personali in un'interfaccia dell'applicazione e queste informazioni includono nome, email, password e foto del volto. Successivamente, non si procede direttamente al salvataggio di queste informazioni nel database ma vengono prima estratte le caratteristiche distintive del volto rappresentate dalla chiave biometrica e soltanto dopo le informazioni vengono memorizzate nel database dell'applicazione qualora l'utente le abbia inserite correttamente.

- Al rilevamento della persona, al momento in cui giunge in una specifica locazione o punto d'interesse indicato dall'utente in fase di creazione dell'attività, da parte per robot si attiva il vero è proprio riconoscimento dove viene generata una nuova chiave biometrica relativa alla foto del volto che l'utente cattura, questa viene confrontata con le chiavi biometriche presenti nel database è se viene trovata una chiave simile a quella nuova generata si procede a verificare se l'identificativo dell'utente che corrisponde alla chiave trovata è uguale all'identificativo dell'utente che ha creato l'attività. In fase di successo viene notificata l'attività all'utente altrimenti si procede prendendo in considerazione altre azioni come tornare alla station base.

Questo è un processo efficace per fornire un'interazione personalizzata tra il robot e gli utenti, migliorando l'esperienza utente e consentendo al robot di svolgere attività specifiche in base all'identità dell'utente.

## 3.2 Progettazione dell'applicazione

Il processo di progettazione svolge un ruolo fondamentale nello sviluppo di qualsiasi applicazione, in quanto consente di definire in modo chiaro e dettagliato l'interfaccia utente e le funzionalità chiave dell'applicazione stessa. Andremo a definire ciò facendo uso di particolari rappresentazioni grafiche, dell'intera applicazione, che prendono il nome di *mock-up*.

I *mock-up* sono una rappresentazione visiva delle schermate e delle interazioni che compongono l'applicazione. Essi sono una versione statica e non interattiva dell'interfaccia utente, ma svolgono un ruolo cruciale nel comunicare l'aspetto e il funzionamento dell'applicazione agli stakeholder o ai membri del team di sviluppo.

Il processo di progettazione con *mock-up* rappresenta un'importante fase preliminare nello sviluppo di qualsiasi applicazione, in quanto consente di definire in modo chiaro e concreto l'aspetto e le funzionalità dell'applicazione stessa. Attraverso l'utilizzo dei *mock-up*, si intende fornire una base solida e ben definita per il successivo sviluppo dell'applicazione, garantendo un prodotto finale di alta qualità e in grado di soddisfare le esigenze degli utenti in modo ottimale.

In questa sezione della presente tesi saranno illustrati i *mock-up* progettati per lo sviluppo dell'applicazione, focalizzandoci sull'usabilità, l'accessibilità e sull'esperienza utente.

### 3.2.1 Schermata “AUTHENTICATION”

La schermata *Authentication* (Figura 1.23) è la prima schermata che si incontra all’avvio dell’applicazione.

Essenziale ed intuitiva, contiene un messaggio di benvenuto e permette all’utente di:

- **Registrarsi al sistema** mediante la schermata di Sing-up : compilando un form con il proprio nome, la propria email, una password e una foto del volto (Figura 1.24).

- **Accesso al sistema** mediante la schermata di Login : compilando un form con email e password con cui l’utente si è registrato (Figura 1.25). Infine, tale schermata, permette di poter cambiare/aggiornare la password del relativo account tramite il pulsante “*Forgot password?*”.

Entrambe le funzioni verranno effettuate solo se se l’utente compila correttamente tutti i campi testuali dei form e in caso di successo il robot viene istruito a notificare l’utente vocalmente, mediante feedback positivo, il relativo successo dell’operazione. In caso di fallimento il robot verrà istruito a notificare vocalmente, mediante feedback negativo, il relativo insuccesso dell’operazione.




| APP_MCK01 | |
|---|---|
| Componente [1] | |
| Tipo | Pulsante |
| Funzione | Permette di chiudere l'applicazione su Temi e va alla schermata "temi's apps" |
| Componente [2] | |
| Tipo | Pulsante |
| Funzione | Apre la schermata APP_MCK02 |
| Componente [3] | |
| Tipo | Pulsante |
| Funzione | Apre la schermata APP_MCK04 |
| Componente [4] | |
| Tipo | Pulsante |
| Funzione | Apre un panello dedicato all'esecuzione delle attività in base all' |

Figura 1.23 : Mock-up Schermata Authentication

| APP_MCK1.1 | | |
|---|---|---|
|  | | |
| | Componente [1] | |
| | Tipo | Pulsate |
| | Funzionalità | Permette di ritornare alla schermata precedente |
| | Componente [2] | |
| | Tipo | Campo testuale |
| | Funzionalità | Apre una dialog che permette di aiutare l'utente nella compilazione |
| | Componente [3] | |
| | Tipo | Campo testuale |
| | Funzionalità | Campo testuale per l'inserimento del nome dell'utente |
| | Componente [4] | |
| | Tipo | Campo testuale |
| | Funzionalità | Campo testuale per l'inserimento dell'email dell'utente |
| | Componente [5] | |
| | Tipo | Campo testuale |
| | Funzionalità | Campo testuale per l'inserimento della password dell'utente |
| | Componente [6] | |
| | Tipo | Pulsate |
| | Funzionalità | Permette di accedere, accettando i permessi, alla fotocamera del dispositivo |
| | Componente [7] | |
| | Tipo | Pulsate |
| | Funzionalità | Apre una dialog che aiuta l'utente a compilare correttamente i campi testuali |

Figura 1.24 : Mock-up Schermata Sing-UP

| APP_MCK1.2 | | |
|---|---|---|
|  | Componente [1] | |
| | Tipo | Pulsate |
| | Funzionalità | Permette di ritornare alla schermata precedente |
| | Componente [2] | |
| | Tipo | Campo testuale |
| | Funzionalità | Apre una dialog che permette di aiutare l'utente nella compilazione |
| | Componente [3] | |
| | Tipo | Campo testuale |
| | Funzionalità | Campo testuale per l'inserimento dell'email dell'utente |
| | Componente [4] | |
| | Tipo | Campo testuale |
| | Funzionalità | Campo testuale per l'inserimento della password dell'utente |
| | Componente [5] | |
| | Tipo | Pulsante |
| | Funzionalità | Permette di accedere alla schermata APP_MCK02 |
| | Componente [6] | |
| | Tipo | Pulsate |
| | Funzionalità | Permette di aprire una dialog per il cambio della password in caso in cui l'utente la smarrisce o la dimentica. |

Figura 1.25 : Mock-up Schermata Login

## 3.2.2 Schermata “HOME”

La schermata *Home* (Figura 1.26) è la prima schermata che si incontra dopo che l’utente ha effettuato correttamente l’accesso.

Permette all’utente di navigare tra le varie schermate in particolare: alla schermata di registrazione del percorso cliccando sul pulsante “*CREATE ROUTE*” ; alla schermata attraverso il quale l’utente potrà scegliere di inserire, cancellare o modificare una locazioni/punto d’interesse sulla mappa cliccando sul pulsante “*OPTIONS LOCATION OR POINT INTEREST*”; alla schermata in cui verranno salvati i vari percorsi presenti nella piattaforma cliccando sul pulsante “*START PATROL*”; alla schermata in cui potranno essere programmate le attività cliccando sul pulsante “*SCHEDULER TASKS*”.




| APP_MCK02 | |
|---|---|
| Componente [1] | |
| Tipo | Pulsate |
| Funzionalità | Permette di ritornare alla schermata precedente |
| Componente [2] | |
| Tipo | Pulsate |
| Funzionalità | Apre la schermata APP_MCK03 |
| Componente [3] | |
| Tipo | Pulsate |
| Funzionalità | Apre la schermata APP_MCK04 |
| Componente [4] | |
| Tipo | Pulsate |
| Funzionalità | Apre la schermata APP_MCK09 |
| Componente [5] | |
| Tipo | Pulsante |
| Funzionalità | Apre la schermata APP_MCK13 |
| Componente [6] | |
| Tipo | Pulsate |
| Funzionalità | Permette all’utente di disconnettersi dal proprio account |

Figura 1.26 : Mock-up Schermata Home

### 3.2.3 Schermata “CREATE ROUTE”

La schermata *Create Route* (Figura 1.27) è la schermata che permette di registrare e salvare un percorso sulla mappa del robot Temi.

Il percorso viene creato facendo in modo che l’utente compila un *form* in cui inserisce un nome al percorso che sta creando e una serie di tappe, pre-salvate sulla mappa interna del robot, che compongono il percorso e il numero di volte in cui l’utente desidera di far navigare il robot lungo il percorso.

Tutti i campi testuali sono soggetti a validazione e sarà valido solo se l’utente ha compilato correttamente tutti i campi e se non sono vuoti tutti o in parte. Nel caso in cui tutti i campi non risultano vuoti e sono validi il robot viene istruito a pronunciare ,vocalmente, mediante feedback positivo il relativo successo dell’operazione. In caso contrario il robot viene istruito a pronunciare, vocalmente, mediante feedback negativo il relativo insuccesso dell’operazione.

APP_MCK03




| Componente[1] | |
|---|---|
| Tipo | Pulsante |
| Funzione | Permette di ritornare alla schermata precedente |
| **Componente[2]** | |
| Tipo | Campo testuale |
| Funzione | Apre una dialog che permette di aiutare l’utente nella compilazione |

Figura 1.27 : Mock-up Schermata Create Route

APP_MCK03




| Componente[3] | |
|---|---|
| Tipo | Campo testuale |
| Funzione | Campo testuale per l'inserimento del nome del percorso |
| Componente[4] | |
| Tipo | FluidSlider |
| Funzione | Barra di scorrimento per indicare il numero di volte che il robot deve navigare lungo il percorso |
| Componente[5] | |
| Tipo | Pulsante |
| Funzione | Permette all'utente di far navigare il robot lungo il percorso all'infinito |
| Componente[6] | |
| Tipo | Widget |
| Funzione | Permette di inserire le locazioni o punti d'interesse che compongono un percorso. |
| Componente[7] | |
| Tipo | Pulsante |
| Funzione | Permette di inserire le locazioni o punti d'interesse che compongono un percorso. |
| Componente[8] | |
| Tipo | Pulsante |
| Funzione | Permette di inserire il percorso nel database del sistema |

Figura 1.27 : Mock-up Schermata Create Route - fine.

### 3.2.4 Schermata “START PATROL”

La schermata *Start Patrol* (Figura 1.28) è la schermata che permette agli di visualizzare una lista contenente tutti i percorsi da egli stesso creati.

Inoltre ogni percorso selezionato dall’utente potrà essere eliminato, modificato, eseguito il che significa far navigare il robot lungo il percorso creato che tra l’altro potrà decidere egli stesso stoppare in qualsiasi momento (Figura 1.29).

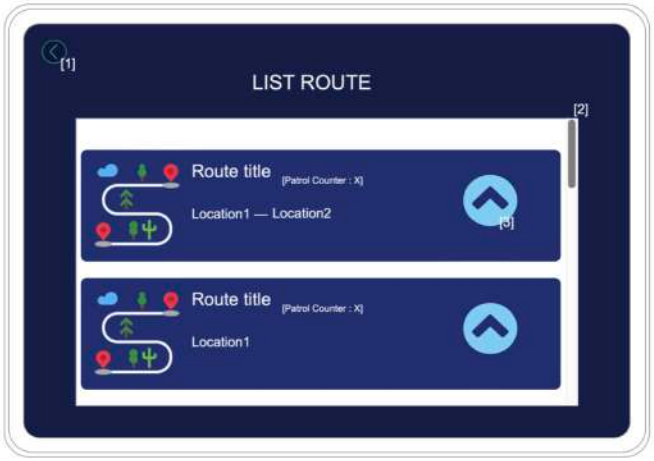
| APP_MCK04 | | |
|---|---|---|
|  | Componente [1] | |
| | Tipo | Pulsate |
| | Funzionalità | Permette di ritornare alla schermata precedente |
| | Componente [2] | |
| | Tipo | Lista |
| | Funzionalità | Contiene tutti i percorsi presenti nel sistema, filtrati in base all’utente che li crea |
| | Componente [3] | |
| | Tipo | Pulsate |
| | Funzionalità | Permette di aprire aprire uno “sportello” dove è possibile effettuare vare operazioni APP_MCK4.1 |

Figura 1.28 : Mock-up Schermata *Scheduler Tasks*

| APP_MCK4.1 | | |
|---|---|---|
|  | Componente [1] | |
| | Tipo | Pulsate |
| | Funzionalità | Permette di eliminare il percorso selezionato |
| | Componente [2] | |
| | Tipo | Pulsate |
| | Funzionalità | Permette di modificare il percorso selezionato |
| | Componente [3] | |
| | Tipo | Pulsate |
| | Funzionalità | Permette di eseguire il percorso selezionato |

Figura 1.29 : Mock-up Schermata Create Route

## 3.2.5 Schermata “OPTIONS ACTIVITY”

La schermata *Options Activity* (Figura 1.30) è la schermata che permette agli utenti, che hanno effettuato correttamente l’accesso di usufruire di ulteriori funzioni implementate nell’applicazione che sono:

1. Inserire una o più attività per il percorso selezionato mediante la schermata “*Insert Activity*”.

2. Visualizzare la propria lista delle attività mediante la schermata “*View List Activity*”.

Nel dettaglio nella schermata *Insert Activity* (Figura 1.31) viene inserita un o più attività da parte dell'utente. Tale operazione consisterà nel compilare un *form* da parte dell'utente che dovrà compilare assegnandogli un nome all'attività, una descrizione, selezionando una locazione (ad esempio cucina, stanza da letto, salone, ufficio) pre-esistenti sulla mappa, inserendo un'attività (ad esempio prenotazione, cena, pranzo, lettura, medicine), indicando una fascia oraria e infine sezionando un giorno in cui il robot dovrà segnalare /ricordare l'attività in questione all'utente.

L'inserimento dell'attività verrà effettuate solo se se l'utente compila correttamente tutti i campi testuali dei form e se lo slot ora-giorno non è occupato. Nel caso di successo il robot viene istruito a notificare vocalmente, mediante feedback positivo il relativo successo dell'operazione. Nel caso di fallimento il robot viene istruito a notificare vocalmente, mediante feedback negativo il relativo insuccesso dell'operazione.

Mentre nella schermata *View List Activity* (Figura 1.32) vengono mostrate la lista delle attività per il percorso selezionato dall'utente, che ha effettuato l'accesso, e ognuna di essa la si può eliminare, modificare o attivare (Figura 1.33).

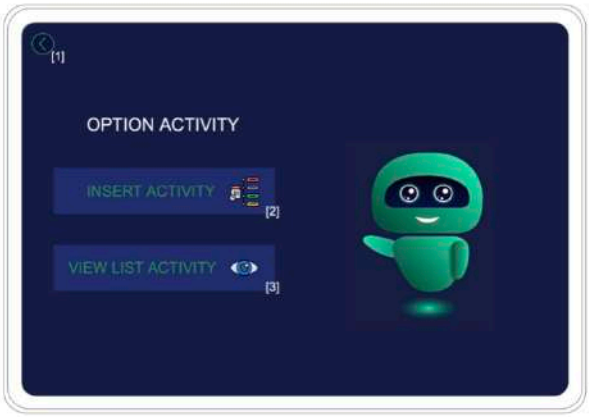
| APP_MCK05 | | |
|---|---|---|
|  | Componente [1] | |
| | Tipo | Pulsate |
| | Funzionalità | Permette di tornare alla schermata precedente |
| | Componente [2] | |
| | Tipo | Pulsate |
| | Funzionalità | Apre la schermata APP_MCK06 |
| | Componente [3] | |
| | Tipo | Pulsate |
| | Funzionalità | Apre la schermata APP_MCK07. |

Figura 1.30 : Mock-up Schermata Options Activity

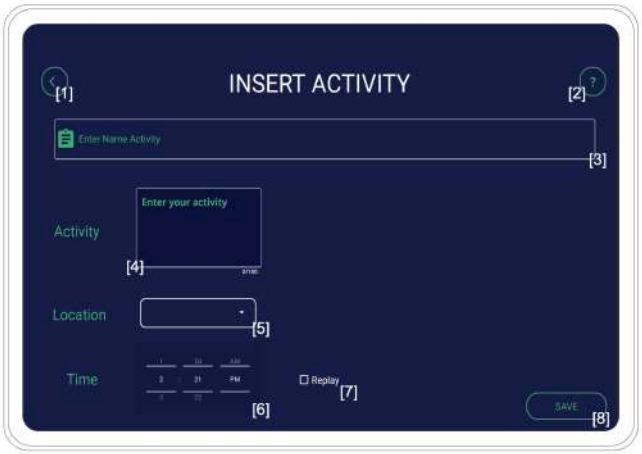
| APP_MCK06 | | |
|---|---|---|
|  | Componente[1] | |
| | Tipo | Pulsante |
| | Funzione | Permette di ritornare alla schermata precedente |
| | Componente[2] | |
| | Tipo | Campo testuale |
| | Funzione | Apre una dialog che aiuta l'utente a compilare il form correttamente |
| | Componente[3] | |
| | Tipo | Campo testuale |
| | Funzione | Campo testuale per l'inserimento del nome dell'attività |
| | Componente[4] | |
| | Tipo | Campo testuale |
| | Funzione | Campo testuale per l'inserimento della descrizione dell'attività |
| | Componente[5] | |
| | Tipo | Spinner |
| | Funzione | Permette di selezionare una locazione o un punto d'interesse salvato sulla mappa |

Figura 1.31 : Mock-up Schermata Insert Activity

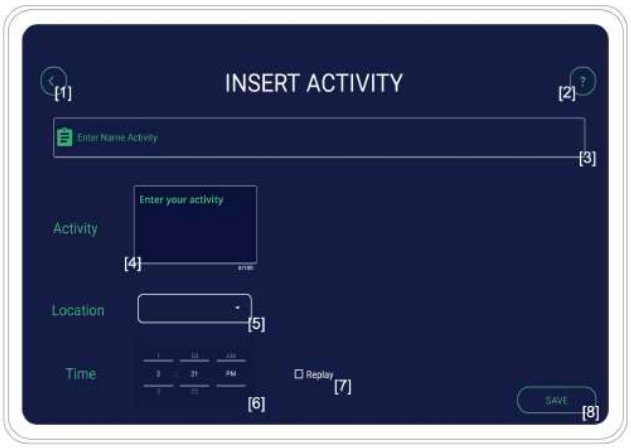
| APP_MCK06 | | |
|---|---|---|
|  | Componente[6] | |
| | Tipo | TimePicker |
| | Funzione | Permette di inserire la fascia oraria |
| | Componente[7] | |
| | Tipo | CheckBox |
| | Funzione | Permette di inserire un giorno della settimana |
| | Componente[8] | |
| | Tipo | Pulsante |
| | Funzione | Permette di inserire l'attività nel database del sistema |

Figura 1.31 : Mock-up Schermata Insert Activity - fine

| APP_MCK07 | | |
|---|---|---|
|  | Componente [1] | |
| | Tipo | Pulsate |
| | Funzionalità | Permette di tornare alla schermata precedente |
| | Componente [2] | |
| | Tipo | Pulsate |
| | Funzionalità | Contiene tutte le attività relative al percorso selezionato dall'utente |
| | Componente [3] | |
| | Tipo | Pulsate |
| | Funzionalità | Apre uno "sportello" per fare diverse operazioni APP_MCK5.1 |
| | Componente [4] | |
| | Tipo | Campo testuale |
| | Funzionalità | Permette di aprire una dialog con la descrizione dell'attività selezionata dall'utente |

Figura 1.32 : Mock-up Schermata View List Activity

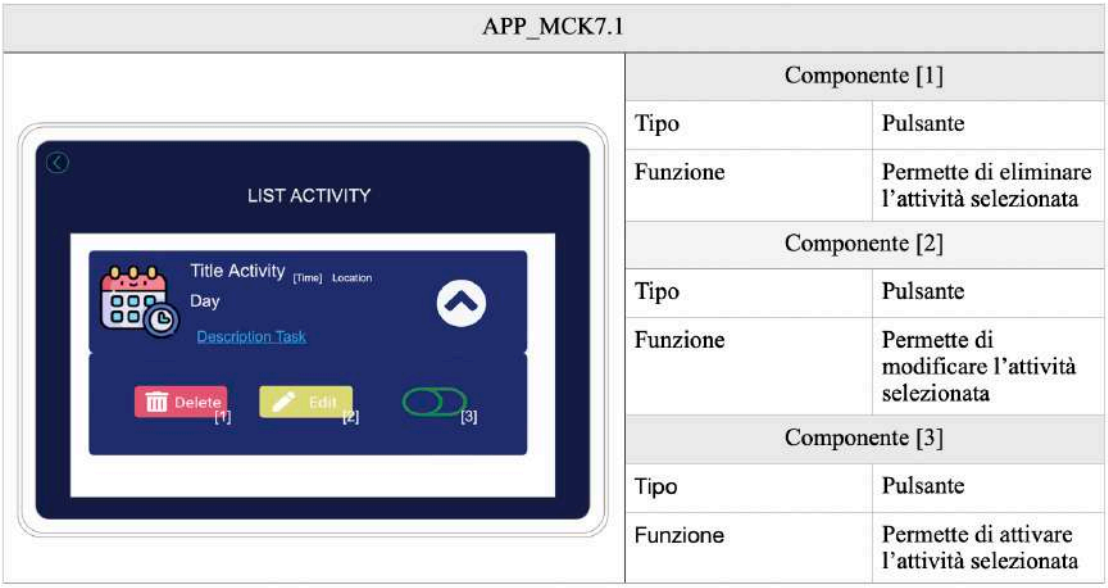



Figura 1.33 : Mock-up Schermata View List Activity

## 3.2.6 Schermata "ACTIVITY NOTIFICATION"

La schermata *Activity Notification* è la schermata (Figura 1.34) che permette al *robot* di notificare l'attività ad un preciso utente.

Quindi una volta attivata l'attività pianificata dall'utente all'ora stabilità il robot va alla ricerca dell'utente e mostra la schermata di riconoscimento (Figura 1.35) se l'utente viene riconosciuto gli ricorda l'attività. In caso contrario va alla ricerca dell'utente nell'ambiente e se lo trova gli notifica l'attività e salva la posizione in cui ha trovato l'utente.

| APP_MCK8.1 | | |
|---|---|---|
|  | Componente[1] | |
| | Tipo | Box |
| | Funzione | Indica l'attività da ricordare all'utente |
| | Componente[2] | |
| | Tipo | Pulsante |
| | Funzione | Utilizzato dall'utente per confermare al robot che ha ricevuto il messaggio |
| | Componente[2] | |
| | Tipo | Pulsante |
| | Funzione | Utilizzato dall'utente per far ritornare il robot alla base dopo che ha confermato la notifica. |

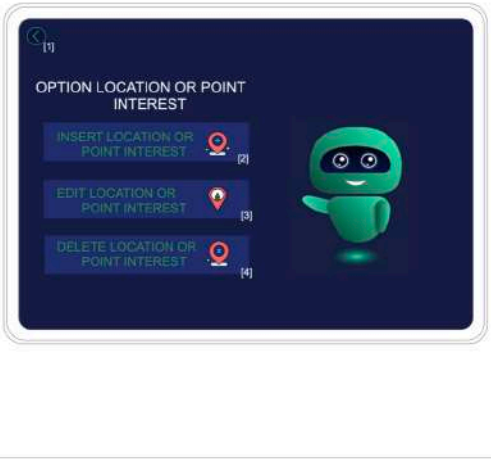
Figura 1.34 : Mock-up Schermata Activity Notification

| APP_MCK08 | | |
|---|---|---|
|  | Componente[1] | |
| | Tipo | Pulsante |
| | Funzione | Permette di accedere alla fotocamera interna del robot |
| | Componente[2] | |
| | Tipo | Pulsante |
| | Funzione | Riconosce l'utente e il creatore dell'attività |

Figura 1.35 : Mock-up Schermata Activity Notification

## 3.2.7 Schermata “Option Location Or Point Interest”

La schermata *Option Location Or Point Interest* (Figura 1.36) è la schermata che permette di inserire, cancellare e modificare una locazione o un punto d’interesse sulla mappa.




| APP_MCK09 | |
|---|---|
| Componente [1] | |
| Tipo | Pulsante |
| Funzione | Permette di ritornare alla schermata precedente |
| Componente [2] | |
| Tipo | Pulsante |
| Funzione | Apre la schermata APP_MCK10 |
| Componente [3] | |
| Tipo | Pulsante |
| Funzione | Apre la schermata APP_MCK11 |
| Componente [4] | |
| Tipo | Pulsante |
| Funzione | Apre la schermata APP_MCK12 |

Figura 1.36 : Mock-up Schermata Option Location Or Point Interest

## 3.2.8 Schermata “Insert Location Or Point Interest”

La schermata *Insert Location Or Point Interest* (Figura 1.37) è la schermata che permette di inserire una locazione o un punto d’interesse sulla mappa.

L’inserimento viene fatto facendo in modo che l’utente si faccia seguire dal robot cliccando il pulsante “*Follow me*” (Figura 1.38) e appena il robot inizia a seguire

l'utente e l'utente si ferma la schermata mostra un campo in cui l'utente inserisce il nome della locazione/punto d'interesse, infine cliccando sul pulsante "*save*" la locazione/punto d'interesse viene salvata direttamente sulla mappa presente nel robot Temi (Figura 1.38).

Inoltre nella schermata stessa viene anche mostrata una lista di tutte le locazioni/punti d'interesse presenti sulla mappa.

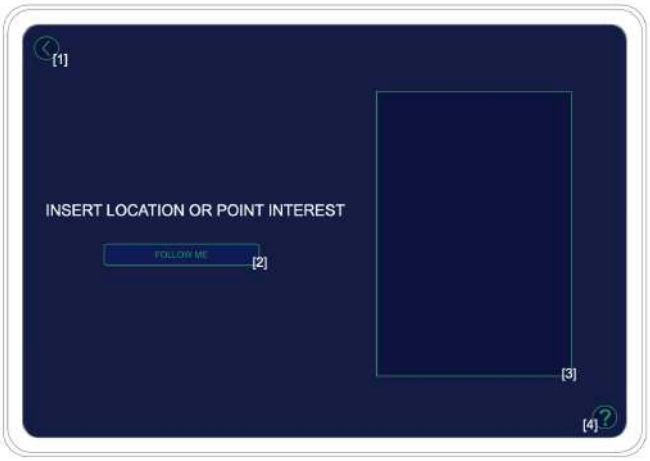
| APP_MCK10 | | |
|---|---|---|
|  | Componente [1] | |
| | Tipo | Pulsante |
| | Funzione | Permette di ritornare alla schermata precedente |
| | Componente [2] | |
| | Tipo | Pulsante |
| | Funzione | Attiva la funzione che permette al robot Temi di seguire l'utente |
| | Componente [3] | |
| | Tipo | Lista |
| | Funzione | Contiene la lista di tutte le locazioni o i punti d'interesse presenti sulla mappa |
| | Componente [4] | |
| | Tipo | Pulsante |
| | Funzione | Apre una dialog che aiuta l'utente ad usare correttamente la schermata APP_MCKQ10 |

Figura 1.37 : Mock-up Schermata Insert Location Or Point Interest

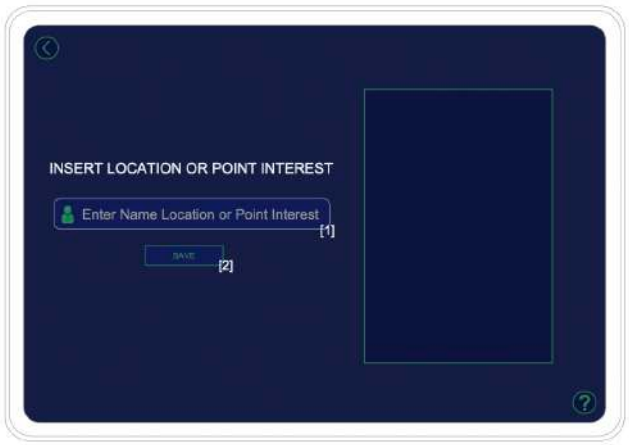
| APP_MCK10.1 | | |
|---|---|---|
|  | Componente [1] | |
| | Tipo | Campo testuale |
| | Funzione | Campo testuale per l'inserimento del nome della locazione o del punto d'interesse |
| | Componente [2] | |
| | Tipo | Pulsante |
| | Funzione | Permette di salvare la posizione corrente del robot |

Figura 1.38 : Mock-up Schermata Insert Location Or Point Interest.

## 3.2.9 Schermata "Delete Location Or Point Interest"

La schermata *Delete Location Or Point Interest* (Figura 1.39) è la schermata che permette di eliminare una locazione o un punto d'interesse sulla mappa.

L'eliminazione viene fatta facendo in modo che l'utente inserisce la locazione/punto d'interesse che desidera eliminare e quando clicca sul pulsante "remove" la locazione viene automaticamente eliminata dalla mappa in Temi.

Inoltre nella schermata stessa viene anche mostrata una lista di tutte le locazioni/punti d'interesse presenti sulla mappa di Temi.

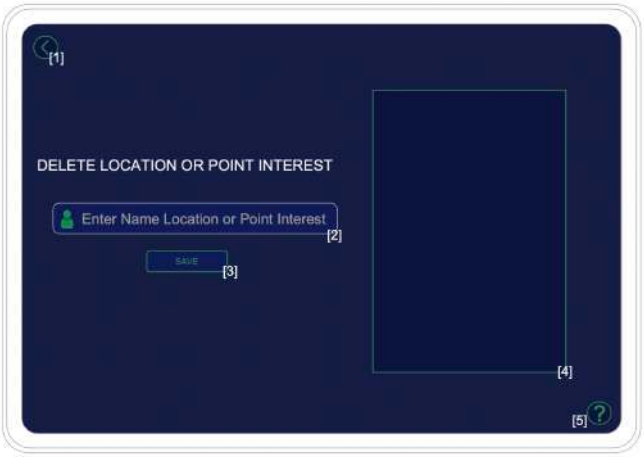
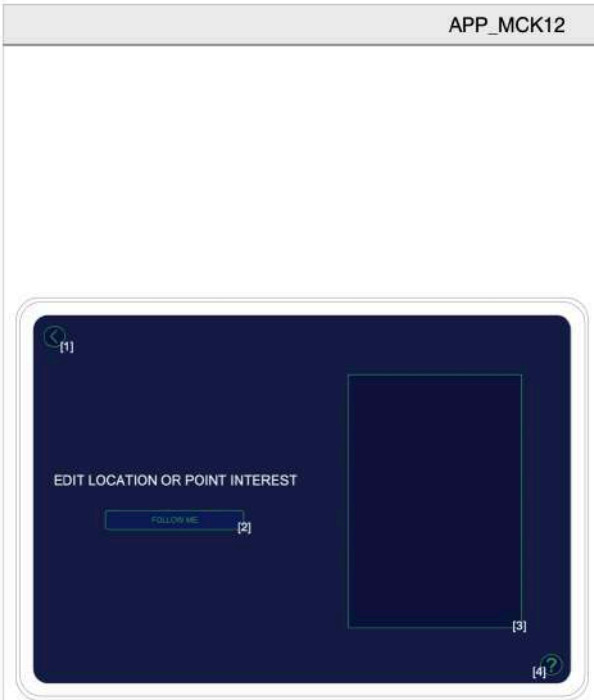
| APP_MCK11 | | |
|---|---|---|
|  | Componente [1] | |
| | Tipo | Pulsante |
| | Funzione | Permette di ritornare alla schermata precedente |
| | Componente [2] | |
| | Tipo | Campo testuale |
| | Funzione | Campo testuale per l'eliminazione della locazione o del punto d'interesse salvato sulla mappa. |
| | Componente [3] | |
| | Tipo | Pulsante |
| | Funzione | Permette di eliminare realmente dalla mappa la locazione o il punto d'interesse |
| | Componente [4] | |
| | Tipo | Lista |
| | Funzione | Contiene la lista di tutte le locazioni o i punti d'interesse presenti sulla mappa del robot Temi |
| | Componente [5] | |
| | Tipo | Pulsante |
| | Funzione | Apre una dialog che aiuta l'utente ad usare correttamente la schermata APP_MCK11 |

Figura 1.39 : Mock-up Schermata Delete Location Or Point Interest

## 3.2.10 Schermata “Edit Location Or Point Interest”

La schermata *Edit Location Or Point Interest* è la schermata (Figura 1.40) che permette di modificare una locazione o un punto d’interesse sulla mappa.

La modificare viene fatta facendo in modo che l’utente si faccia seguire dal robot cliccando il pulsante “*Follow me*” (Figura 1.40) e appena il robot inizia a seguire l’utente e l’utente si ferma in un punto già salvato sulla mappa, la schermata mostra un campo in cui, l’utente , inserisce il nuovo nome della locazione/punto d’interesse, infine cliccando sul pulsante “*save*” la locazione/punto d’interesse va a sovrascrivere la locazione/punto d’interesse precedentemente salvato sulla mappa con la nuova locazione/punto d’interesse (Figura 1.41).




| APP_MCK12 | |
|---|---|
| Componente [1] | |
| Tipo | Pulsante |
| Funzione | Permette di ritornare alla schermata precedente |
| Componente [2] | |
| Tipo | Campo testuale |
| Funzione | Attiva la funzione che permette al robot Temi di seguire l’utente |
| Componente [3] | |
| Tipo | Lista |
| Funzione | Contiene tutte le locazioni o i punti d’interesse presenti sulla mappa del robot Temi |
| Componente [4] | |
| Tipo | Pulsante |
| Funzione | Apre una dialog che iuta l’utente ad usare correttamente la schermata APP_MCK12 |

Figura 1.40 : Mock-up Schermata Edit Location Or Point Interest

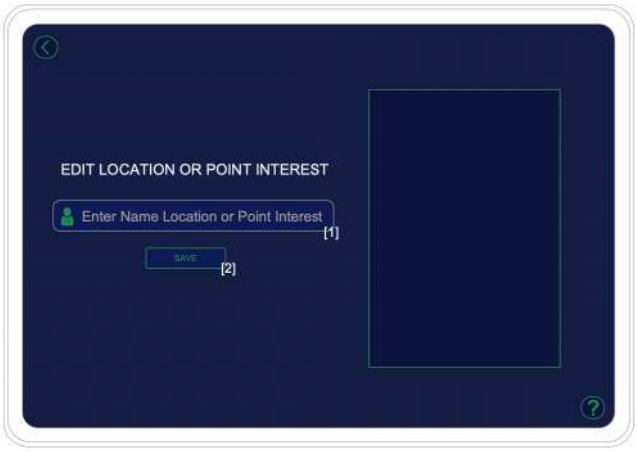
| APP_MCK12.1 | | |
|---|---|---|
|  | Componente [1] | |
| | Tipo | Campo testuale |
| | Funzione | Campo testuale per l'inserimento del nome della locazione o del punto d'interessee |
| | Componente [2] | |
| | Tipo | Pulsante |
| | Funzione | Permette di modificare realmente sulla mappa del robot Temi locazioni o punti d'interesse |

Figura 1.41 : Mock-up Schermata Edit Location Or Point Interest - fine

## 3.2.11 Schermata “Execution tasks”

La schermata *Execution tasks* è la schermata (Figura 1.42) in cui vengono eseguiti i task programmati da tutti gli utenti, registrati al sistema, in base all’orario, in particolare, viene eseguito prima quello più vicino all’orario corrente e poi gli altri man mano.

| APP_MCK13 | | |
|---|---|---|
|  | Componente [1] | |
| | Tipo | Campo testuale |
| | Funzione | Permette di ritornare alla schermata precedente. |
| | Componente [2] | |
| | Tipo | Lista |
| | Funzione | Contiene tutte le attività programmate dall’utente. |

Figura 1.41 : Mock-up Schermata Execution tasks

# 4. Architettura dell’applicazione

In questo capitolo analizzeremo architettura del applicazione fornendo una descrizione dettagliata dell'architettura dell'applicazione, delineando come l'applicazione è strutturata e come sono organizzati i suoi componenti principali. Inoltre descriveremo il modello di progettazione dell'architettura dell’applicazione basato s MVVM (Model-View-ViewModel) dove andremo a delineare come i dati, la logica aziendale e l'interfaccia utente sono separati e interagiscono tra loro.

## 4.1 Implementazione dell’applicazione

### 4.1.1 Architettura Esterna

L’architettura esterna dell’applicazione, oggetto della presente tesi, è stata realizzata tramite l’architettura a “*three-tier*”[12] (Figura 1.41) , che si articola in tre differenti moduli:

- l’interfaccia utente;
- la logica dell’applicazione;
- l’archiviazione dei dati;

[12] https://www.ibm.com/it-it/topics/three-tier-architecture

L'accesso agli stessi è sviluppato e mantenuto sotto forma di moduli indipendenti. Questo consente che in casi di variazioni ad esempio nei requisiti o nella tecnologia, è possibile aggiornare o sostituire qualsiasi dei tre livelli senza dover modificare gli altri.

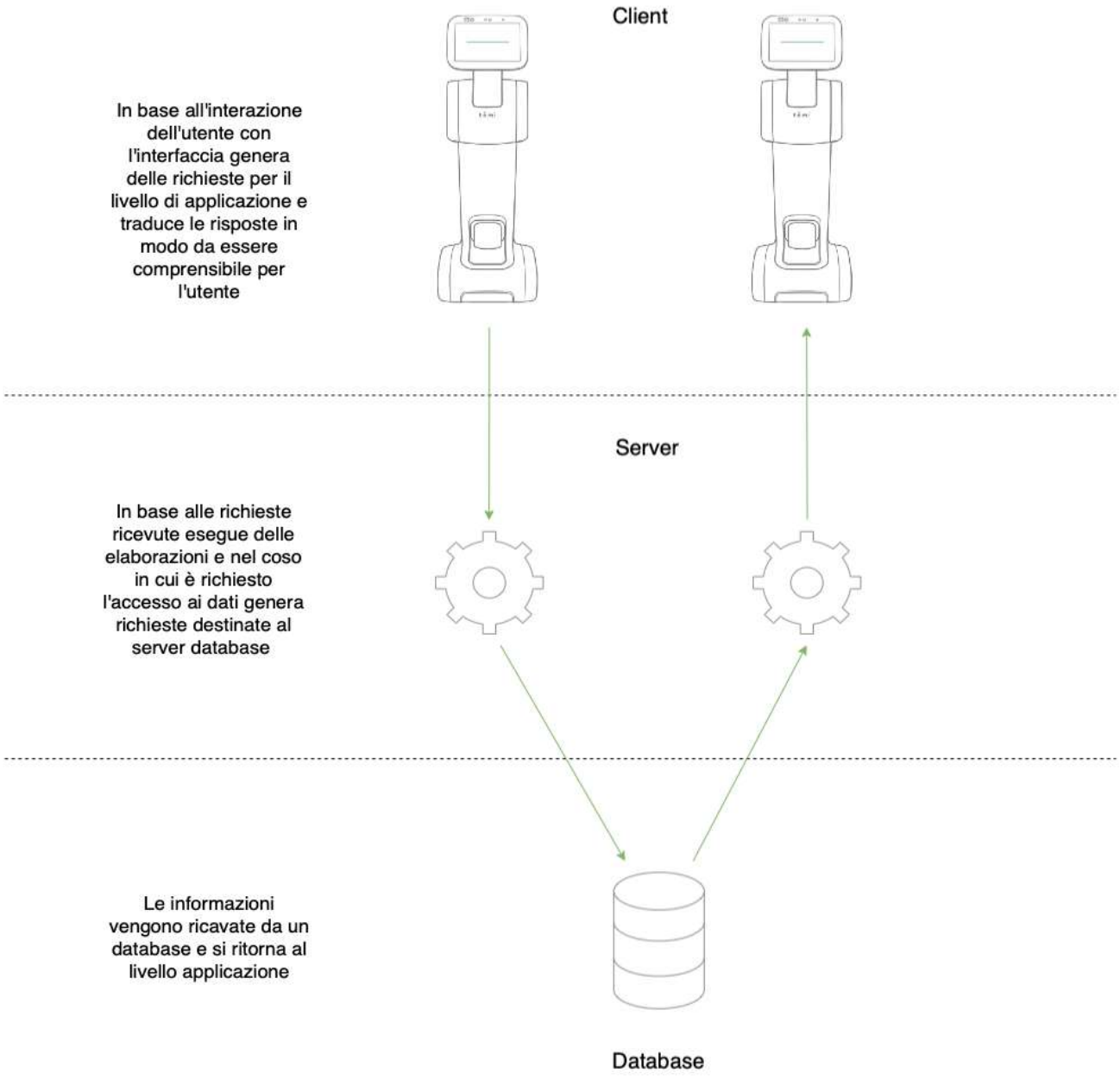



Figura 1.41 : Architettura esterna dell'applicazione

Come si nota dalla Figura 1.41, il funzionamento dell'architettura esterna dell'applicazione è il seguente: il client invia richieste al back-end, viene interpretato ciò che è contenuto nelle richieste, si eseguono eventuali elaborazioni e si generano altre richieste che vengono destinate al motore del database. Appena il livello intermedio riceve i dati dal database, il risultato in output viene reso comprensibile all'utente nella forma più appropriata.

### 4.1.2 Architettura Client

Il *client* dell'applicazione, oggetto della presente tesi, è stato sviluppato su piattaforma Android Studio. Questa scelta è stata guidata non solo dalla compatibilità con il sistema operativo del robot in quanto il *Tablet* del robot Temi è progettato su sistema operativo *Android,* ma anche dall'*SDK,* messa appunto dall'azienda *Robotemi Inc.,* che è stata realizzata in A*ndroid studio* facendo uso di linguaggio J*ava/Kotlin*.

Per l'implementazione dell'applicazione si è scelto di utilizzare un linguaggio *Object Oriented*, in particolare *Java*, con l'ausilio del design pattern "*Model View ViewModel*".

Il *Model View ViewModel* è un *Design pattern*[13] architetturale attraverso il quale è possibile trarre vantaggio in termini di prestazioni, leggibilità e modularità del codice e si è preferito in quanto si adatta meglio all'ambiente *Android Studio*.

[13] https://it.wikipedia.org/wiki/Design_pattern

Composto da tre moduli (Figura 1.42) indipendenti:

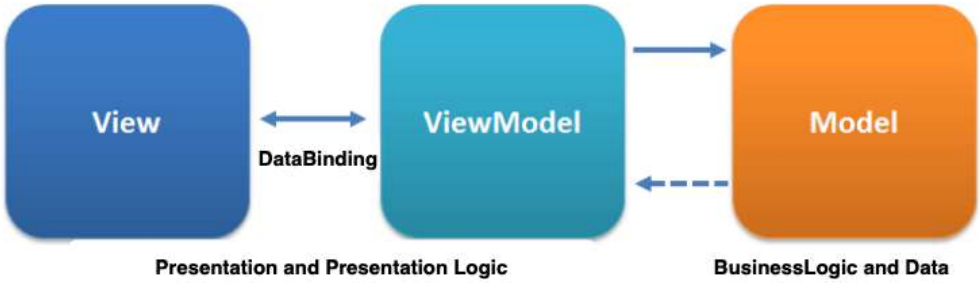



Figura 1.42 : Struttura del MVVM [6]

***Model***

Rappresenta l'insieme di classi, strutture dati e algoritmi che contengono le politiche di alto livello.

***View***

È costituito dall'insieme di classi, strutture dati e algoritmi che interagiscono con il *ViewModel* per visualizzare i dati esposti e passano al *ViewModel* eventuali azioni da parte dell'utente che necessitano di elaborazione locale o remota.

***ViewModel***

È il componente intermediario tra il *Model* e la *View*. Tramite il *Model* si ottengono i dati nativi che vengono trasformati in dati pronti per essere visualizzati e presentati all'utente.

Il meccanismo principale del *ViewModel* è il data binding dove le modifiche ai dati apportate dall'utente attraverso la View vengono automaticamente riportate nel *ViewModel* e viceversa, le modifiche che avvengono ai dati del *ViewModel* vengono automaticamente rappresentate nella *View*.

Il *ViewModel* è completamente indipendente dalla *View*. Più viste possono avere dipendenze dallo stesso *ViewModel*, ma il *ViewModel* non dipende da nessuno.

Infine, MVVM gode di vantaggi e svantaggi:

Vantaggi :

1. **Separazione delle responsabilità** : MVVM favorisce una chiara separazione delle responsabilità tra le diverse componenti dell'applicazione. Il modello rappresenta i dati e la logica di business, la vista si concentra sull'interfaccia utente e il *viewModel* gestisce la logica di presentazione e il *binding* tra *Model* e *View*. Ciò semplifica la manutenzione e l'evoluzione dell'applicazione.

2. **Testabilità** : la separazione delle responsabilità favorisce la testabilità. I test del *viewModel* possono essere scritti in modo indipendente dalla *View* e dal *Model*, consentendo test più focalizzati e rapidi.

3. **Riutilizzo della logica di presentazione** : la logica di presentazione all'interno dei *viewModel* può essere riutilizzata in diverse parti dell'applicazione o in diverse versioni dell'interfaccia utente.

4. **Binding dei dati** : MVVM sfrutta il binding dei dati per sincronizzare automaticamente i dati tra *Model* e *View*. Questo semplifica notevolmente l'aggiornamento dell'interfaccia utente in risposta ai cambiamenti nel *Model* e riduce la necessità di codice *boilerplate* per l'aggiornamento manuale dell'UI.

5. **Scalabilità** : l'architettura MVVM facilita la scalabilità dell'applicazione poiché la separazione delle responsabilità semplifica la gestione dei componenti e il loro sviluppo indipendente.

Tuttavia gode di svantaggi, anche se minimi :

1. **Complessità iniziale** : l'implementazione di MVVM può richiedere una maggiore quantità di codice rispetto a pattern meno strutturati, aumentando la complessità iniziale dell'applicazione.

2. **Apprendimento** : l'apprendimento di MVVM può richiedere tempo, specialmente per gli sviluppatori meno familiari con il pattern o con il framework utilizzato.

3. ***Overhead*** : in alcuni casi, l'implementazione completa del MVVM potrebbe sembrare eccessiva per applicazioni semplici o piccoli progetti.

4. **Possibili cattive implementazioni** : se non implementato correttamente, MVVM potrebbe portare a confusione nella separazione delle responsabilità o alla creazione di *viewM*odel troppo complessi.

Nello sviluppo dell'applicazione si è preferito utilizzare il pattern MVVM per i suoi molteplici vantaggi che come abbiamo definito in precedenza si riflettono principalmente in termini di struttura, separazione delle responsabilità e testabilità delle applicazioni.

In dettaglio la i componenti dell'architettura del MVVM impiegati nello sviluppo sono i seguenti (Figura 1.43) :

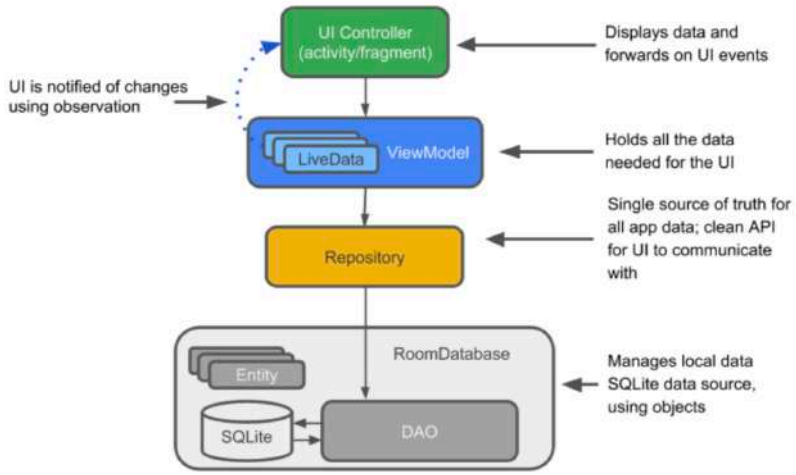



Figura 1.43 : Componenti dell'architettura MVVM Dell'applicazione [7]

I componenti che utilizzeremo all'interno dell'applicazione sono elencati di seguito con una spiegazione dettagliata [8]:

- **LiveData**: LiveData è una classe di titolari di dati che può essere osservata. Conserva e memorizza nella cache l'ultima versione dei dati e avvisa l'osservatore ogni volta che i dati vengono aggiornati o modificati. Live Data gestisce automaticamente tutto questo poiché è a conoscenza dei cambiamenti dello stato del ciclo di vita rilevanti durante l'osservazione[14].

[14] https://it.wikipedia.org/wiki/Observer_pattern

- **ViewModel**: ViewModel funge da centro di comunicazione tra il repository e l'interfaccia utente. L'interfaccia utente non deve più preoccuparsi dell'origine dei dati. Le istanze ViewModel sopravvivono alla ricreazione di *activity*/*fragment*.

- **Repository**: Repository è una classe utilizzata principalmente per gestire più fonti di dati.

- **Entity**: Entity è una classe annotata che viene utilizzata per descrivere una tabella di database quando ci si lavora con Room.

- **Room Database**: il database delle stanze è una versione improvvisata del database SQLite. Semplifica il lavoro del database e funge da punto di accesso al database SQLite sottostante. Room utilizza DAO per inviare query al database SQLite.

- **DAO**: DAO è un oggetto di accesso ai dati che viene utilizzato per mappare le query SQL alle funzioni.

# 5. Implementazione

In questo capitolo, esploreremo in dettaglio le scelte implementative e le soluzioni che sono state adottate per raggiungere gli obiettivi prefissati nel contesto del progetto.

## 5.1 Dettagli Implementativi

Le scelte progettuali per ottenere una buona implementazione sono state:

- Android Studio in quanto il tablet di Temi è progettato su sistema operativo *android* e ciò ha semplificato l'integrazione e massimizzato la compatibilità. Inoltre l'SDK prodotta da *Robotemi Inc.* è realizzata in *android studio* di conseguenza è possibile creare applicazioni che fanno uso di funzionalità avanzate del robot in modo semplice, rapido e sicuro.

- SDK di Temi robot in quanto permette di integrare funzionalità specifiche, come abilità vocali, rilevamento dell'utente, mobilità e molto altro, del robot personalizzandole all'interno dell'applicazione.

- Paradigma Object-Oriented, in quanto permette una facile gestione e manutenzione di progetti anche di grandi dimensioni e favorisce la modularità e il riuso di codice

- Python 3, un linguaggi di programmazione ad oggetti, scelto per la libreria "*face_recognition*" che fornisce un'interfaccia *user-friendly* per il riconoscimento dei volti e offre funzionalità avanzate come l'individuazione dei volti nelle immagini e il confronto tra volti per il riconoscimento.

### 5.1.1 Autenticazione

Nello sviluppo del software l'utente può usufruire di molteplici funzioni offerte dal sistema a patto che sia autenticato al sistema.

La fase di autenticazione si articola in due fasi :

1. L'utente deve registrarsi , se non registrato a-priori, all'applicazione
2. L'utente deve effettuare l'accesso

L'interfaccia con il quale l'utente si registra all'applicazione si presenta con un form in cui l'utente dovrà inserire nome, email, password che viene cifrata al momento dell'inserimento e una foto del volto.

Qualora l'utente inserisce correttamente tutti i campi, non li lascia vuoti, la foto ha un volto e inserisce email e password con un formato valido in particolare:

- Email con dominio come "esempio@live.it"
- Password contenente almeno almeno otto carattere, almeno un carattere maiuscola, almeno un carattere minuscolo e almeno una cifra numerica

il robot comunica all'utente che è stato salvato con successo. Tale notifica fatta all'utente dal robot viene effettuata istruendolo, il robot, a pronunciarla vocalmente e visualizzandola sullo schermo del robot. Invece, in caso di insuccesso il robot notifica, allo stesso modo, l'utente con un messaggio che indica l'insuccesso dell'operazione.

Quando l'utente viene salvato correttamente nel database, inoltre, oltre alle sue informazioni che inserisce su interfaccia ad ogni utente verrà fatto corrispondere un identificativo univoco che lo identificherà nel sistema.

Per quanto riguarda invece l'accesso, interfaccia relativa a tale operazione si presenta con una schermata in cui l'utente dovrà inserire l'email e la password utilizzate in fase di registrazione. In tal caso nel momento in cui l'utente inserisce su interfaccia email e password questi dati saranno soggetti a controlli ossia viene controllato che non siano vuoti, che siano validi e che siano presente nel database dell'applicazione e tali controlli vengono superati il sistema passa ad una nuova schermata dell'applicazione altrimenti il robot comunica all'utente l'errore verificato. Tal notifica fatta all'utente dal robot viene effettuata istruendolo, il robot, a pronunciarla vocalmente e visualizzandola sullo schermo del robot.

Infine la schermata di accesso da la possibilità all'utente di cambiare la password del proprio account a patto che l'email sia presente nel database dell'applicazione.

### 5.1.2 Creazione e Visualizzazione dei percorsi

Se l'utente fa effettuata l'accesso al sistema correttamente, tra le diverse operazioni di cui può usufruire messe a disposizione vi è quella della creazione del percorso.

L’utente tramite una schermata può programmare e personalizzare uno o più percorsi, ciò potrà essere fatto inserendo una serie di informazioni come il nome del percorso, il numero di volte che l’utente deve “pattugliare” lungo il percorso e una o più locazioni o punti d’interesse queste ultime informazioni le ricaviamo dalla mappa presente all’interno del robot Temi. Una volta aver inserito tutte le informazioni viene controllato dal sistema che i campi non risultino vuoti. A tal punto se tutto è stato inserito correttamente dall’utente allora il percorso viene salvato nel database dell’applicativo e il robot notifica il successo dell’operazione all’utente tramite messaggio vocale e tramite un messaggio testuale.

Nel momento che viene salvato nel database, al percorso corrisponderà non solo l’identificativo univoco del percorso ma anche l’identificativo dell’utente che ha creato il percorso in modo tale da filtrare i percorsi e far in modo che ogni utente abbia la propria lista contenente solo i propri percorsi creati.

Altra schermata disponibile nel sistema è la visualizzazione della lista di percorsi, tale schermata contiene e permette di visualizzare tutti i percorsi creati dall’utente. Ovviamente in tale schermata troviamo solo i percorsi creati dall’utente che ha effettuato l’accesso, ciò è stato fatto filtrando tutti i percorsi disponibili nell’applicazione in base all’identificativo univoco dell’utente. Inoltre per ognuno dei percorsi presenti in questa lista è possibile eliminarlo, modificarlo e risalvarlo ed eseguirlo ossia far navigare il robot lungo il percorso e facendogli attraversare tutte le locazioni o i punti d’interesse inseriti in fase di creazione del percorso.

### 5.1.3 Inserimento locazioni o punti d'interesse

Ulteriore operazione progettata nel sistema è l'inserimento di una nuova locazione o di un nuovo punto d'interesse. L'idea alla base dell'implementazione di tale funzione è : l'utente si fa seguire dal robot e quando si ferma l'utente, in una particolare posizione anche il robot si ferma e rende visibile su interfaccia un campo testuale che permette all'utente di dar un nome al punto in cui il robot si è fermato. In tal caso viene controllato che il campo sia vuote e se tale controllo viene superato il robot notifica all'utente che la posizione è stata salvata correttamente altrimenti il robot notifica all'utente un messaggio relativo all'insuccesso dell'operazione. Ovviamente tale notifiche del robot all'utente vengono effettuate vocalmente dal robot e contemporaneamente anche da un messaggio testuale.

Per la realizzazione di tale idea sono stati utilizzati due metodi offerti dall'SDK del robot temi:

- ***beWithMe()*** : Utilizzato per richiamare manualmente la modalità *follow, dove l*a modalità F*ollow* è lo stato in cui il robot Temi cerca una persona che gli sta di fronte e una volta trovata si blocca e segue il suo movimento finché non viene detto diversamente.

- ***saveLocation()*** : Utilizzato per salvare una nuova posizione in particolare individua il robot Temi nella posizione corrente, le coordinate della posizione vengono estratte e passate automaticamente nella richiesta.

In particolare la finzione saveLocation() viene utilizzata nella seguente funzione:

```
1.  private void button_save_location(){
2.   String insert_location_or_pointInterest =  insert_pointInterest.getEditText().getText().toString().trim();
3.   if(insert_location_or_pointInterest.isEmpty()){
4.         robot.speak(TtsRequest.create("Please fill any empty",true));
5.   } else {
6.       saveLocation(insert_pointInterest);
7.       navController.navigate(R.id.action_InsertLocationOrPointInterest_to_OptionLocationFragment);
8.         }
9.     }
```

In cui si ricava il valore inserito dall'utente su interfaccia, ossia il nome che l'utente associa alla locazione, si verifica se il campo di inserimento è vuoto. Se il campo è vuoto, il robot pronuncia un messaggio chiedendo all'utente di compilare il campo altrimenti chiamiamo il metodo "saveLocation" per salvare la posizione o il punto di interesse e dopo aver salvato i dati, naviga verso il fragment "OptionLocationFragment".

### 5.1.4 Eliminazione locazione o punto d'interesse

Ulteriore operazione progettata nel sistema è l'eliminazione di una locazione o di un punto d'interesse pre-esistente sulla mappa del robot Temi.

Per tale operazione si è utilizzato un metodo dell'SDK di temi ossia "deleteLocation()" che viene utilizzato per eliminare una locazione dalla mappa interna del robot Temi. Tale funzione è stata utilizzata richiamandola in un'altra funzione:

```
1.  public void deleteLocation(View view) {
2.    String remove_location_or_pointInterest = remove_pointInterest.getEditText().getText().toString().trim();
3.    if (remove_location_or_pointInterest.isEmpty()) {
4.         robot.speak(TtsRequest.create("Please fill any empty", true));
5.    } else {
```

```
6.          String locations = remove_pointInterest.getEditText().getText().toString().toLowerCase().trim();
7.          boolean result = robot.deleteLocation(locations);
8.           if (result) {
9.             robot.speak(TtsRequest.create("I've successfully remove the " + locations + " location or point interest.", true));
10.            navController.navigate(R.id.action_DeleteLocationOrPointInterest_to_OptionLocationFragment);
11.          } else {
12.              robot.speak(TtsRequest.create("Remove the " + locations + " location or point interest failed.", true));
13.          }
14.      }
15.  }
```

Tale metodo controlla se l'utente ha inserito una posizione o un punto di interesse da eliminare. Se il campo di inserimento è vuoto, il robot pronuncia un messaggio di avviso ciò viene fatto dal robot vocalmente e tramite un messaggio testuale. Se il campo contiene dati, il metodo chiama il metodo **deleteLocation()** del robot per tentare di rimuovere la posizione o il punto di interesse specificato. Successivamente, il robot comunica l'esito dell'operazione all'utente e, se la rimozione è riuscita, naviga verso un altra schermata.

### 5.1.5 Modifica locazione o punto d'interesse

Nello sviluppo l'operazione di modifica di una locazione o di un punto d'interesse viene effettuato con la stessa idea e metodi dell'inserimento di una locazione o di un punto (3.2.3.3 Inserimento locazioni o punti d'interesse) d'interesse solo quando si chiama la funzione "saveLocation()" su una posizione già presente sulla mappa interna del robot Temi il nome della nuova locazione va a sovrascrivere il vecchio nome della locazione inserito in precedenza in quella posizione.

### 5.1.6 Inserimento e Visualizzazione attività

Altra operazione possibile dall'utente offerta dal sistema è l'inserimento di una o più attività, tale funzione viene effettuata riempiendo un form in cui l'utente dovrà inserire il nome dell'attività, una descrizione dell'attività, selezionare una locazione, inserire uno slot temporale ed infine selezionare uno o più giorni della settimana. Una volta inseriti i vari campi l'attività verrà salvata nel database dell'applicazione.

Ovviamente tutti i campi testuali presenti nella schermata sono soggetti a validazione/ controlli, i controlli che vengono effettuati sono molteplici tra cui:

- Campi non devono essere vuoti
- Lo slot temporale non deve essere già in uso in giorni in cui già c'è un'attività

Una volta che tali controlli sono superati il robot notifica il successo dell'operazione non solo tramite messaggio vocale ma anche tramite  messaggio testuale altrimenti notifica all'utente il relativo messaggio d'insuccesso.

Inoltre quando l'attività viene salvata nel database del sistema oltre alle varie informazioni inserite dall'utente su interfaccia andiamo ad assegnarli anche un identificativo univoco e l'identificativo univoco dell'utente che ha creato l'attività in modo tale che ogni utente abbia una lista con solo le proprie attività e non possa accede a quelle di altri account utenti.

Altra schermata disponibile nel sistema è la visualizzazione della lista delle attività, tale schermata contiene e permette di visualizzare tutte le attività create dall'utente. Ovviamente in tale schermata le attività sono filtrate in base all'identificativo univoco

dell'utente che ha effettuato l'accesso. Inoltre per ogni attività presenti in questa lista è possibile eliminarla, modificarla e attivare l'attività facendo in modo che il robot si reca alla posizione stabilità e se rileva l'utente notifica l'attività altrimenti viene riprogrammata automaticamente dopo 5 minuti.

### 5.1.7 Notifica attività

Nello sviluppo l'attività verrà notificata dalla classe *SearchUserService,* una classe di servizio che verrà attivato dal *TaskBroadcastReceiver* tramite il metodo *startTaskService()* quando arriva l'ora di inizio di un'attività salvata e attiva.

La struttura che ha reso possibile la realizzazione di quanto descritto sopra è la seguente :

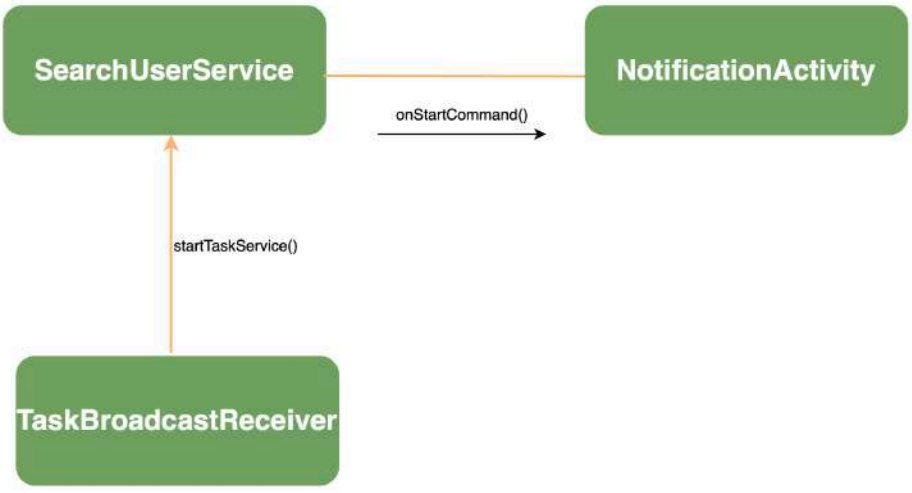



Figura 1.44 : Componenti dell'implementazione della notifica dell'attività

Questo servizio farà in modo che il robot cercherà l'utente nelle località o nel punto d'interesse inserita dall'utente in fase di creazione dell'attività e una volta giunto alla località o nel punto d'interesse attiverà la funzione di rilevamento. Se non viene rilevato nessun utente, il robot cerca l'utente tra le varie locazioni presenti sulla mappa. Se trova qualcuno, si avvicinerà alla persona ed la avviserà che forse ha un'attività programmata , infine mostrerà una schermata per il riconoscimento del volto, a tal punto se l'utente viene riconosciuto ed è il creatore dell'attività viene mostrata l'attività altrimenti il robot torna alla docking station.

Il riconoscimento dell'utente, ovviamente, dal punto di vista implementativo viene fatto facendo in modo che l'utente accede alla videocamera del suo dispositivo e si cattura l'immagine del suo volto. Tale immagine, quando viene cliccato il pulsante di salvataggio, viene inviata al server Python che controlla se nella foto vi è la presenza del volto oppure no. Se nella foto non vi è almeno un volto, il robot notifica tale questione all'utente vocalmente in caso contrario viene generata la chiave biometrica corrispondente all'immagine. Una volta generata la chiave biometrica viene confrontata Con quelle presenti nel database e dopo aver controllato che l'identificatore dell'utente a cui corrisponde la chiave biometrica è lo stesso che ha creato il task, il robot notifica l'attività all'utente tramite un messaggio testuale altrimenti l'utente viene notificato con un messaggio da parte del robot in cui viene avvisato che non è l'utente e torna alla docking station.

Per far ciò sono stati utilizzati diversi metodi dell'SDK di temi robot:

- ***startDetection()*** : avvia il processo di rilevamento dell'utente quando si trova nella posizione specificata.

- ***detecting()*** : gestisce il rilevamento effettivo. Imposta il robot in modalità di rilevamento e quindi fa ruotare il robot in incrementi di 45 gradi fino a quando non rileva un utente o compie una rotazione completa di 360 gradi. Quando un utente viene rilevato o il robot completa la rotazione, vengono intraprese altre azioni.

- ***turnBy()*** : progettato per far ruotare il robot di 45 gradi in un certo senso se durante il rilevamento non ha individuato neanche una persona

- ***navigateToLocation()*** : implementato per far muovere il robot verso una posizione specifica, ma solo se la posizione è valida. Se la posizione non è valida, viene registrato un messaggio di errore.

- ***requestPermissionIfNeeded(…)*** : richiede le autorizzazioni e i controlli, se questi vengono concessi

- ***interacting()*** : gestisce le interazioni dell'utente con il robot. In particolare, arrestare il movimento del robot e disattiva la modalità di rilevamento quando un utente sta interagendo.

- ***onUserInteraction(…)*** : è un ascoltatore dell'SDK di Temi e verrà chiamato quando l'utente tocca il display.

- ***searchForUserElsewhere()*** : cerca l'utente tra tutte le locazioni o i punti d'interesse sulla mappa interna del robot nel momento in cui non rileva alcun utente alla posizione indicata alla creazione del task.

- ***onDetectionStateChanged(…)*** : è un ascoltatore dell'SDK di Temi e riceve lo stato del rilevamento. Se viene rilevata una persona, il Temi dice “ciao, probabilmente hai un’attività programmata", avvia anche la classe NotificationActivity e termina il servizio *SearchingUserService*.

- ***onGoToLocationStatusChanged(…)*** : è un ascoltatore dell'SDK di Temi. Controlla se il robot è arrivato nella posizione selezionata. Quando il robot arriva nella posizione chiama *detecting*().

Inoltre, il criterio di scelta dell’attività da notificare è fatto in base all’orario corrente ,ovviamente solo se l’attività è attivata, il che significa che se ad esempio sono le “15:00” ed ho :

- “Attività1” alle “15:23”
- “Attività2” alle “15:26”

Verrà eseguita prima “Attività1” e poi “Attività2”.

# 6. Testing

In questo capitolo verrà illustrato il processo di validazione del progetto realizzato e descritto nel capitolo Approccio Proposto. L'obiettivo sarà quello di descrivere il processo di valutazione e test dell'applicazione sviluppata, focalizzandoci sull'interazione uomo-robot e in particolare su aspetti legati all'usabilità dell'applicazione prodotta.

## 6.1 Descrizione dei test

Il test prevede uno studio di progettazione partecipativa[15] che consiste nel far immergere l'utente in scenari tipici della vita quotidiana, a supporto di ciò è stato creato uno spazio in cui gli utenti possono muoversi liberamente e interagire con l'ambiente circostante e il robot Temi. Inoltre, ogni utente ha partecipato ad una sessione con un facilitatore.

Nel contesto di questa simulazione di vita quotidiana, gli utenti affrontano una serie di attività quotidiane, ognuna delle quali implica l'interazione con il robot e l'utilizzo dell'applicazione.

[15] https://it.wikipedia.org/wiki/Progettazione_partecipativa

Alcuni esempi di queste attività includono:

- **Registrarsi e accedere all'applicazione** : Gli utenti devono creare un account e successivamente effettuare l'accesso per utilizzare le diverse funzionalità dell'applicazione.

- **Creare Percorsi** : Gli utenti devono creare uno o più percorsi per il robot Temi. Questi percorsi possono essere successivamente modificati, eliminati o eseguiti dal robot.

- **Inserire, eliminare e aggiornare locazioni o punti d'interesse** : Gli utenti devono inserire nuove locazioni o punti di interesse nel sistema. Possono anche eliminarli o modificarli in seguito.

- **Inserire attività** : Gli utenti devono aggiungere attività da far notificare al robot Temi all'ora e giorno corrente.

- **Eliminazione e aggiornamento delle attività** : Dopo aver creato un'attività, gli utenti devono eliminarla o apportare aggiornamenti in base alle loro esigenze.

- **Notifiche e Promemoria** : Gli utenti, dopo aver programmato una o più attività, possono attivarle. In tal caso, l'applicazione invia notifiche e promemoria agli utenti per varie attività, come appuntamenti, riunioni o l'assunzione di farmaci. Gli utenti possono interagire con il robot Temi per ricevere e gestire queste notifiche.

La sperimentazione, in conclusione, consente nel valutare l'efficacia dell'applicazione e l'interazione uomo-robot.

## 6.2 Valutazione

Al test in totale hanno partecipato 2 persone e analizzando i dati partecipativi, risulta che l'età minima è 29 anni, l'età massima è 32 anni.

Degli intervistati risulta che 1 è di sesso femminile e 1 è di sesso maschile, inoltre, riguardo la nazionalità si ha: 1 indiana e 1 italiana. A livello occupazionale, risulta che 1 è studente e 1 è ricercatore.

Riguardo le esperienze pregresse con robot, entrambe le due persone hanno dichiarato di aver avuto qualche tipo di interazione con un robot.

Per effettuare il test di usabilità, sono state prese in considerazione le seguenti attività. Nella seguente lista sono indicate l'attività svolta, un tempo massimo preventivato per lo svolgimento dell'attività ed un esempio delle loro risposte:

- Compito 1. Registrati alla piattaforma [10 minuti]

Studente dice di essersi sentito guidato durante tutta l'operazione, soprattutto grazie ai messaggi d'aiuto. Per la ricercatrice l'osservatore ha notato che durante lo svolgimento di questo compito ha riscontrato alcune difficoltà dovute alla registrazione del volto. Nonostante ciò tutti hanno completato l'attività con successo.

- Compito 2. Accedere alla piattaforma [3 minuti]

Ricercatrice e Studente dicono di essersi sentiti guidato durante tutta l'operazione, soprattutto grazie ai messaggi d'aiuto e ai messaggi d'errore .Tutti hanno completato l'attività con successo.

- Compito 3. Creazione percorso [7 minuti]

Entrambi i valutatori dicono di essersi sentito guidati grazie alla schermata molto intuitiva e ai messaggi d'aiuto.  Sono stati in grado di completare quest'attività con successo nel tempo preventivato.

- Compito 4. Esecuzione percorso [2 secondi]

Entrambi i valutatori sono stati in grado di completare quest'attività con successo nel tempo preventivato.

- Compito 5. Eliminazione percorso [2 secondi]

Entrambi i valutatori sono stati in grado di completare quest'attività con successo nel tempo preventivato.

- Compito 6. Modifica percorso [5 minuti]

Entrambi i valutatori sono stati in grado di completare quest'attività con successo nel tempo preventivato.

- Compito 7. Inserimento punto d’interesse o locazione [10 minuti]

Entrambi i valutatori sono stati in grado di completare quest’attività con successo nel tempo preventivato tuttavia entrambi pensavano che la lista con le locazioni fosse cliccabile ma dopo che l’osservatore gli ha spiegato il funzionamento hanno trovato logico che vengano mostra tutte le locazioni presenti sulla mappa.

- Compito 8. Cancellazione  punto d’interesse o locazione [5 minuti]

Entrambi i valutatori sono stati in grado di completare quest’attività con successo nel tempo preventivato tuttavia entrambi pensavano che la lista con le locazioni fosse cliccabile ma  dopo che l’osservatore gli ha spiegato il funzionamento hanno trovato logico che vengano mostra tutte le locazioni presenti sulla mappa.

- Compito 9. Modifica punto d’interesse o locazione [5 minuti]

Entrambi i valutatori sono stati in grado di completare quest’attività con successo nel tempo preventivato. Tuttavia entrambi pensavano che la lista con le locazioni fosse cliccabile ma dopo alle spiegazioni dell'osservatore hanno compreso correttamente come funziona il sistema e hanno portato a termine entro con successo l’attività.

- Compito 10. Inserimento attività [10 minuti]

Entrambi i valutatori sono stati in grado di completare quest’attività con successo ma non nel tempo preventivato, hanno impiegato qualche minuti in più relativo all’inserimento dei dati ma entrambi ritengono che l’interfaccia sia semplice da usare

non solo per il design semplice e intuitivo ma anche per la segnalazioni in caso di errore nell'inserimento dell'attività .

- Compito 11. Eliminazione attività [2 secondi]

Entrambi i valutatori sono stati in grado di completare quest'attività con successo nel tempo preventivato.

- Compito 12. Modifica attività [4 minuti]

Entrambi i valutatori sono stati in grado di completare quest'attività con successo nel tempo preventivato.

- Compito 13. Attivazione/Disattivazione dell'attività [2 secondi]

Entrambi i valutatori sono stati in grado di completare quest'attività con successo nel tempo preventivato.

- Compito 14. Esecuzione dell'attività

Entrambi i valutatori non hanno utilizzato questa funzione hanno solo inserito l'attività .

- Compito 15. Riconoscimento dell'utente con successiva notifica [5 minuti]

Entrambi i valutatori sono stati in grado di completare quest'attività con successo nel tempo preventivato.

## 6.3 Risultati

I risultati sono stati ottenuti intervistando e ponendo delle domande dopo l'iterazione con il robot. Di seguito vengono mostrati i risultati :

- Registrazione all'applicazione : semplice (2 su 2)
- Accesso all'applicazione : semplice (2 su 2)
- Creazione percorso : semplice (2 su 2)
- Creazione attività : semplice (2 su 2)
- Notifica attività : accurata (2 su 2)
- Riconoscimento : molto accurato (2 su 2)
- Velocità navigazione : abbastanza veloce (2 su 2)
- Applicazione intuitiva : molto intuitiva (2 su 2)
- App facile da usare : si, l'ho trovata molto facile da usare (2 su 2)

Analizzando i dati raccolti nella domanda relativa all'iterazione tra l'utente e l'applicazione è emerso che gli utenti hanno trovato l'applicazione molto soddisfacente, su una scala da 1 a 10, pari a 8.

Analizzando i dati raccolti nella domanda “vorresti aggiungere delle funzionalità o migliorare qualcosa” è emerso che:

- Utente 1 : suggerisce di indicare su interfaccia il requisito per la creazione della password. Quindi indicare su interfaccia che la password debba avere :
    - Minimo 8 caratteri.
    - Almeno una lettera maiuscola.
    - Almeno una lettera minuscola.
    - Almeno un numero.

- Utente 2 : suggerisce di adattare il volume della voce del robot al rumore della stanza o dell’ambiente in modo che il robot risulti udibile in modo chiaro anche in ambienti rumorosi e non solo tranquilli, inoltre suggerisce di fare una cosa analoga anche per la la luminosità dello schermo.

# 7. Conclusioni e lavori futuri

In questa tesi, è stata studiata e sviluppata una soluzione per migliorare l'interazione tra gli esseri umani e i robot.

L’obiettivo è stato implementare un sistema di gestione dello scheduler per le attività quotidiane degli utenti e dunque è stato creato un sistema di interfaccia utente efficace con lo scopo di rendere più semplice e intuitiva la pianificazione e gestione delle attività quotidiane. In particolare è stata realizzata un’applicazione multi-utente attraverso il quale dopo una prima fase di registrazione e accesso, egli possono accedere a diverse funzioni come:

1. creare un percorso personalizzato contenete le tappe desiderate dagli utenti stessi.

2. visualizzare una lista contenente tutti i percorsi che a loro volta possono essere eliminati, modificati ed eseguiti in qualsiasi momento. Inoltre quando si parla di “far eseguire un percorso al robot” si intende far navigare il robot Temi lungo le locazioni o punti d’interesse, reali, salvati sulla mappa a-priori.

3. Inserire; modificare e eliminare locazioni o punti d’interesse sulla mappa del robot Temi.

4. Inserire e visualizzare una lista di attività dove una volta che l'utente ha registrato l'attività se attivata all'ora corrente il robot va alla ricerca dell'utente nella locazione è qualora lo trova avvia il riconoscimento che in caso di esito positivo gli mostra l'attività altrimenti l'attività viene ritardata di 10 minuti.

Durante lo sviluppo dell'applicazione sono state affrontate diverse sfide relative alla progettazione, all'implementazione e all'usabilità.

Tuttavia, in fase di test dall'analisi dei dati raccolti si è riscontrato che effettivamente l'applicazione ha un impatto positivo sull'organizzazione delle attività quotidiane degli utenti, migliorando l'efficienza e la gestione del tempo.

Nonostante i risultati significativi raggiunti in questo progetto di tesi, ci sono diversi sviluppi futuri e miglioramenti. Ecco alcuni potenziali lavori futuri:

- Migliorare il tempo impiegato dal robot per rilevare l'utente e notificargli l'attività all'ora e al giorno ricorrente;

- Aggiungere una funzionalità tale che se l'utente nella posizione non viene rilevato si calcola la posizione più vicina se l'utente viene rilevato si salva la nuova posizione in cui è stato trovato l'utente calcolando una stima di probabilità che successivamente si trovi in quella particolare posizione.

- Aggiungere allarmi consistenti e non solo notifica vocale nel momento in cui l'utente viene rilevato e bisogna notificare l'attività (come ad esempio aumentare illuminazione dello schermo nel momento in cui il robot va in una stanza abbastanza buia e deve notificare qualcosa all'utente, oppure aumentare

la voce nel momento in cui il robot si trova in una stanza con maggior rumore, etc)

# Bibliografia e Sitografia

[1] Joe Saunders, Dag Sverre Syrdal, Kheng Lee Koay, Nathan Burke, and Kerstin Dautenhahn*, Senior Member, IEEE.* "Teach Me–Show Me"—End-User Personalization of a Smart Home and Companion Robot. IEEE TRANSACTIONS

[2] Bengisu Cagiltay, Hui-Ru Ho, Joseph E Michaelis, Bilge Mutlu. Investigating Family Perceptions and Design Preferences for an In-Home Robot

[3] Architettura rete di Temi robot, https://temi-guide.readthedocs.io/en/latest/network-admin/02_network-architecture.html

[4] Mappa creata con tecnica SLAM in Temi robot, https://www.youtube.com/watch?v=NIoNLHY9-m4

[5] Room Library, https://codingwithmitch.com/blog/sqlite-and-the-room-persistence-library/

[6] Model View ViewModel, https://en.wikipedia.org/wiki/Model–view–viewmodel

[7] Room, LiveData, and ViewModel, https://google-developer-training.github.io/android-developer-fundamentals-course-concepts-v2/unit-4-saving-user-data/lesson-10-storing-data-with-room/10-1-c-room-livedata-viewmodel/10-1-c-room-livedata-viewmodel.html

[8] Florina Muntenescu, Android Architecture Patterns Part 3: Model-View-ViewModel, 4 Novembre 2016, https://medium.com/upday-devs/android-architecture-patterns-part-3-model-view-viewmodel-e7eeee76b73b